WYNIK

Aplicación de aprendizaje musical con reconocimiento de notas en tiempo real

Prototipo de escritorio en Python + Kivy/KivyMD con arquitectura preparada para Android

---

Anexo IV – Documentación Técnica de Programación

Especificación técnica, estructura del repositorio y manual del programador



Asignatura: Taller Práctico Laboral II: Desarrollo de Soluciones

Carrera: Informática · **Institución:** Universidad Católica de Temuco (UCT)

Profesor: Henry Nelson Seguel Quilodran

Integrantes

Segundo Richard Nicolás Mansilla Lleucún

Sebastián Eduardo Contreras Vergara

Benjamín Ignacio Saavedra Mercado

Javiera Alejandra Celia Muñoz Toledo

Fecha de entrega: 04/11/2026 · **Versión:** 3.0

Ficha del documento y control de versiones

La Tabla 1 identifica el documento según la plantilla del CCII y la Tabla 2 registra su evolución.

Tabla 1. Ficha del documento (plantilla CCII-N2016-02)

Campo	Valor
Proyecto	Wynik – aplicación de práctica musical con reconocimiento de notas en tiempo real
Documento	Anexo IV – Documentación Técnica de Programación
Código del documento	WYNIK-ANX-IV
Versión y fecha	3.0 · 04/11/2026
Asignatura	Taller Práctico Laboral II: Desarrollo de Soluciones
Profesor	Henry Nelson Seguel Quilodran
Institución y lugar	Universidad Católica de Temuco (UCT) · Temuco, Chile
Autores y roles	Segundo Richard Nicolás Mansilla Lleucún: Scrum Master y desarrollador Sebastián Eduardo Contreras Vergara: desarrollador y tester Benjamín Ignacio Saavedra Mercado: desarrollador y tester Javiera Alejandra Celia Muñoz Toledo: desarrolladora y tester
Código documentado	Repositorio github.com/MansillsRich/Wynik-app, rama main, commit 358dcaf (03/11/2026)
Aprobación	Sujeta a revisión del profesor de la asignatura
Clasificación	Uso académico
Palabras clave	Python, Kivy, KivyMD, MusicXML, YIN, detección de altura, JSON, unittest, Ruff, pydoc

Tabla 2. Control de versiones

Versión	Fecha	Autor	Descripción del cambio
2.0	03/11/2026	Sebastián Contreras	Versión previa: estructura de carpetas, stack, módulos, pruebas (13/13), plataformas y matriz de funcionalidades.
3.0	04/11/2026	Equipo Wynik	Actualización integral contrastada con el código (commit 358dcaf): identificación del documento, autoría y roles; índices automáticos; estructura del repositorio, dependencias y licencias; algoritmo de detección de notas y persistencia JSON; interfaz con capturas anotadas; manual del programador (instalación, pruebas, cobertura, pydoc, Ruff y depuración); plataformas, prueba con piano real y estado de Android; trazabilidad, glosario, bibliografía en APA 7 y declaración de uso de inteligencia artificial.

Índice

1. Introducción 7
1.1 Propósito 7
1.2 Alcance 7
1.3 Destinatarios 7
1.4 Normas y referencias aplicadas 7
1.5 Convenciones 8
2. Lenguajes y entorno de desarrollo 9
2.1 Lenguajes 9
2.2 Framework de interfaz 9
2.3 Herramientas y entornos verificados 9
3. Organización del proyecto 11
3.1 Estructura del repositorio 11
3.2 Configuración y dependencias 13
3.3 Librerías de terceros y licencias 14
3.4 Código fuente por paquete 15
3.5 Archivos KV 20
3.6 Algoritmo de detección de notas 20
3.6.1 Captura y paso entre hilos 20
3.6.2 Estimación de altura (YIN) 20
3.6.3 Segmentación en eventos 21
3.6.4 Conversión de frecuencia a nota y tolerancia en cents 21
3.6.5 Sincronización con el tempo 21
3.6.6 Parámetros efectivos 22
3.6.7 Verificación con señal sintética 22
3.7 Persistencia JSON 23
3.8 Recursos gráficos 25
4. Interfaz de la aplicación 26
4.1 Menú principal 28
4.2 Importación de partitura 29
4.3 Selección de partitura (Mis partituras) 30
4.4 Preparación de la práctica (instrumento, modo y tempo) 31
4.5 Ajustes generales 33
4.6 Pantalla de práctica 35
4.7 Práctica en curso: nota correcta, incorrecta y desafinada 36
4.8 Resultados e historial 39
4.9 Vista simplificada 41
5. Manual del programador 42
5.1 Requisitos previos 42
5.2 Instalación paso a paso 42
5.3 Ejecución 42
5.4 Documentación del código 43
5.5 Pruebas automatizadas 44
5.6 Calidad de código con Ruff 45
5.7 Depuración 46
5.7.1 Registros 46
5.7.2 Probar sin micrófono 46
5.7.3 Problemas típicos 46
5.8 Plataformas 47
5.8.1 Prueba en macOS con piano real 47
5.8.2 Pruebas en Windows 11 y Fedora Linux 49

5.8.3 Android..... 49
6. Estado de funcionalidades y trazabilidad..... 51
6.1 Mecanismos de control de calidad aplicados..... 51
6.2 Matriz de trazabilidad..... 51
7. Criterios de aceptación y exclusiones..... 53
7.1 Criterios de aceptación del MVP..... 53
7.2 Exclusiones del MVP..... 53
8. Definiciones y abreviaturas..... 54
9. Bibliografía..... 55
Firmas..... 56
Declaración de uso de inteligencia artificial..... 56

Lista de figuras

Figura 1. Árbol del repositorio a nivel de archivo (commit 358dcaf)....11
Figura 2. Dependencias entre paquetes y con las librerías externas....16
Figura 3. Flujo de datos: micrófono → captura → detección → comparación → puntaje → interfaz → JSON....16
Figura 4. Navegación entre pantallas y diálogos....27
Figura 5. Menú principal con la última partitura importada....28
Figura 6. Diálogo «Importar partitura» con el archivo de ejemplo seleccionado....29
Figura 7. Pantalla «Mis partituras» con las dos partituras importadas....30
Figura 8. «Preparar la práctica», parte superior....31
Figura 9. «Preparar la práctica», parte inferior: modo, tempo, notación y vista....32
Figura 10. Ajustes: micrófono, ruido de fondo y afinación....33
Figura 11. Ajustes: evaluación, volumen y vista....34
Figura 12. Práctica preparada (estado «Lista»)....35
Figura 13. Cuenta previa («Prepárate… 2»)....37
Figura 14. Nota correcta: C4 a +0 cents (✓ verde)....37
Figura 15. Nota incorrecta: F4 cuando se esperaba D4 (× rojo)....38
Figura 16. Nota desafinada: F4 a +35 cents (≈ ocre, cuenta como acierto)....38
Figura 17. Diálogo «Práctica terminada»: 6 de 8 notas correctas (75 %)....39
Figura 18. Pantalla «Progreso» con la sesión registrada....40
Figura 19. Práctica con la vista simplificada....41
Figura 20. Documentación generada con pydoc para el módulo audio.estimador_yin....43
Figura 21. Salida de python -m unittest discover -s tests -v en Linux....44
Figura 22. Salida de ruff check . --statistics (Ruff 0.17.3)....45
Figura 23. Pruebas automatizadas en macOS: 13 pruebas, OK....48
Figura 24. macOS con piano real, primer intento: Do♯1 cuando se esperaba La4, marcado como error....48
Figura 25. macOS con piano real, segundo intento: La4 correcto a −18 cents....48
Figura 26. Pruebas automatizadas en Windows 11: 13 pruebas, OK, con Python 3.12.3 (AMD64)....49
Figura 27. Pruebas automatizadas en Fedora Linux: 13 pruebas, OK, con Python 3.14.10 (x86_64)....49

Lista de tablas

Tabla 1. Ficha del documento (plantilla CCII-N2016-02)....2
Tabla 2. Control de versiones....2
Tabla 3. Correspondencia entre la estructura de referencia (Aikén: Android, Java y Firebase) y Wynik....8
Tabla 4. Herramientas y entornos utilizados....10
Tabla 5. Estructura del repositorio a nivel de archivo, con la responsabilidad de cada archivo.....11
Tabla 6. Dependencias directas: versión declarada, fijada e instalada....14
Tabla 7. Librerías de terceros y sus licencias....15
Tabla 8. Paquete audio/: clases y funciones principales....17
Tabla 9. Paquete nucleo/: clases y funciones principales....18
Tabla 10. Paquete servicios/: clases y funciones principales....19
Tabla 11. Paquete vistas/: clases principales....19
Tabla 12. Archivos KV y reglas que definen....20
Tabla 13. Clasificación de una nota detectada frente a la esperada (nucleo/musica.py)....21
Tabla 14. Parámetros del detector calculados con las clases del código (evidencias/parametros_deteccion.txt)....22
Tabla 15. Resultado de la cadena de detección con entrada simulada (escala de Do mayor, piano, 48 kHz)....23
Tabla 16. Archivos de datos locales....23
Tabla 17. Paleta principal de la interfaz (vistas/tema.py)....25
Tabla 18. Elementos del menú principal....28
Tabla 19. Elementos del diálogo de importación....29
Tabla 20. Elementos de «Mis partituras»....30
Tabla 21. Elementos de «Preparar la práctica» (parte superior)....31
Tabla 22. Elementos de «Preparar la práctica» (parte inferior)....32
Tabla 23. Elementos de «Ajustes» (parte superior)....33
Tabla 24. Elementos de «Ajustes» (parte inferior)....34
Tabla 25. Elementos de la pantalla de práctica....36
Tabla 26. Elementos del resumen final....39
Tabla 27. Elementos de la pantalla de progreso....40
Tabla 28. Elementos de la vista simplificada....41
Tabla 29. Requisitos previos por sistema operativo....42
Tabla 30. Pruebas por archivo (resultado del 04/11/2026)....44
Tabla 31. Cobertura de sentencias por paquete (coverage.py 7.17.1, 04/11/2026)....45
Tabla 32. Avisos de Ruff por regla (ruff check . --statistics)....45
Tabla 33. Problemas típicos, causa y solución....46
Tabla 34. Estado de cada plataforma con su evidencia....47
Tabla 35. Estado de Wynik en Android....50
Tabla 36. Mecanismos de control de calidad....51
Tabla 37. Trazabilidad funcionalidad → implementación → prueba → evidencia....52
Tabla 38. Criterios de aceptación y su cumplimiento....53
Tabla 39. Definiciones y abreviaturas....54

1. Introducción

1.1 Propósito

Este anexo reúne la información técnica que necesita un programador para comprender, instalar, ejecutar, probar, depurar y mantener **Wynik**, una aplicación en Python con Kivy/KivyMD que carga partituras MusicXML, escucha al usuario por el micrófono y le indica en tiempo real si toca las notas correctas.

El contenido se contrastó con el código fuente del repositorio y con la salida de los comandos ejecutados el 04/11/2026. Las salidas completas se conservan en `docs/anexo_iv/evidencias/`, junto con los scripts que las generan en `docs/anexo_iv/`.

1.2 Alcance

El anexo describe la versión del repositorio `Wynik-app` en la rama `main`, commit `358dcaf` (03/11/2026). Cubre los lenguajes y herramientas, la estructura del repositorio a nivel de archivo, la configuración y las dependencias, el código fuente por paquete, los archivos KV, el algoritmo de detección de notas, la persistencia JSON, los recursos gráficos, la interfaz con capturas anotadas, el manual del programador y la trazabilidad entre funcionalidades, código y pruebas. No incluye el análisis de requisitos ni el manual del usuario final.

En su estado actual, Wynik es un **prototipo de escritorio** que se ejecuta con Python en macOS, Windows y Linux. Su arquitectura separa el análisis de audio y la lógica musical de la interfaz, lo que permite llevarlos a Android; la versión para Android, empaquetada con Buildozer, se encuentra en desarrollo (apartado 5.8).

1.3 Destinatarios

El documento está dirigido a los integrantes del equipo, al profesor de la asignatura Taller Práctico Laboral II: Desarrollo de Soluciones y a quienes continúen el desarrollo. Se asume conocimiento básico de Python, de la línea de comandos y de Git.

1.4 Normas y referencias aplicadas

De la plantilla del CCII (norma CCII-N2016-02, v1.0) se tomaron solo los apartados aplicables a un anexo: ficha del documento y control de versiones, normas y referencias, mecanismos de control de calidad (apartado 6.1), trazabilidad requisito → implementación → prueba (apartado 6.2) y definiciones y abreviaturas (apartado 8). La estructura y el nivel de detalle toman como referencia el Anexo IV del proyecto Aikén (Rodríguez Ramírez, 2019), adaptado a las tecnologías de Wynik según la Tabla 3. La bibliografía sigue el formato APA 7.

Tabla 3. Correspondencia entre la estructura de referencia (Aikén: Android, Java y Firebase) y Wynik

Referencia (Aikén)	Equivalente en Wynik	Apartado
AndroidManifest.xml + build.gradle	`pyproject.toml` + `requirements-escritorio.in` / `.txt`	3.2
Paquetes Java	Paquetes Python `audio/`, `nucleo/`, `servicios/`, `vistas/`	3.4
res/layout (XML)	Archivos KV: `vistas/wynik.kv` y `vistas/componentes.kv`	3.5
res/drawable, mipmap	Dibujo vectorial con el canvas de Kivy, fuente de iconos y paleta de `vistas/tema.py` (sin mapas de bits)	3.8
JavaDoc	Docstrings + HTML generado con pydoc	5.4
Firebase (Auth, Firestore, Storage)	Persistencia local en tres archivos JSON, sin inicio de sesión ni servicios remotos	3.7
Depurar y probar (AVD / dispositivo físico)	Ejecución en escritorio, pruebas unittest, entrada simulada sin micrófono y estado de la versión para Android	5.5–5.8

1.5 Convenciones

- Las rutas son relativas a la raíz del repositorio (`Wynik-app/`); el código, los comandos y los nombres de archivo van en `monoespaciado`.
- **Validado** indica que la ejecución se comprobó con fecha, entorno y evidencia (captura o salida de comando). **Soportado por dependencias** indica que las dependencias declaran soporte para la plataforma, sin una ejecución registrada.

2. Lenguajes y entorno de desarrollo

2.1 Lenguajes

- **Python 3.13 o superior** (`requires-python = ">=3.13"` en `pyproject.toml`; Ruff configurado con `target-version = "py313"`). Implementa la lógica musical, el procesamiento de audio, los servicios y la persistencia.
- **Lenguaje KV** de Kivy, para declarar la interfaz por separado de la lógica: `vistas/wynik.kv` (pantallas) y `vistas/componentes.kv` (componentes).
- **MusicXML** (`.musicxml`, `.xml` y `.mxl`) como formato de entrada de las partituras, leído con music21.
- **JSON** para guardar ajustes, historial de partituras y progreso.

2.2 Framework de interfaz

Kivy 2.3.1 aporta la ventana, los eventos, el `ScreenManager`, el canvas de dibujo, el reloj (`Clock`) y la reproducción de audio (`SoundLoader`). **KivyMD 2.0.0** se usa de forma acotada: `MDApp` como clase base de la aplicación y su tema, la fuente de iconos «Icons» (`kivymd.icon_definitions.md_icons`), las fuentes Roboto y `MDSnackbar` para los avisos emergentes. Los componentes visibles (botones, opciones, tarjetas, barras) son propios y están en `vistas/componentes.py` y `vistas/componentes.kv`; las pantallas derivan de `kivy.uix.screenmanager.Screen`.

2.3 Herramientas y entornos verificados

La Tabla 4 lista las herramientas y los entornos utilizados, con su versión y la evidencia correspondiente.

Tabla 4. Herramientas y entornos utilizados

Herramienta	Versión	Uso	Evidencia
Python (macOS, Apple Silicon)	3.13.16 (Homebrew)	Desarrollo; entorno virtual `../.venv`; 13/13 pruebas	`.venv/pyvenv.cfg`; Figura 23
Python (Windows 11)	3.12.3 (64 bits, AMD64)	Desarrollo; 13/13 pruebas	Figura 26
Python (Fedora Linux)	3.14.10 (x86_64)	Desarrollo; 13/13 pruebas	Figura 27
Python (Linux x86_64, Ubuntu 24.04)	3.13.18	Instalación, pruebas, cobertura, capturas y pydoc	`evidencias/entorno.txt`
pip / uv	pip 26.2.1 (macOS); uv (Ubuntu)	Instalación de dependencias; uv generó `requirements-escritorio.txt`	Metadatos del `.venv`; cabecera del `.txt`
Git y GitHub	Remoto `github.com/MansillsRich/Wynik-app`, rama `main`, 11 commits	Control de versiones	`git log`, `git remote -v`
unittest	Biblioteca estándar	Pruebas automatizadas (13)	`evidencias/unittest.txt`
Ruff	0.17.3 (el proyecto exige ≥ 0.5.0)	Análisis estático y orden de importaciones	`evidencias/ruff.txt`
coverage.py	7.17.1	Medición de la cobertura de las pruebas	`evidencias/cobertura.txt`
pydoc	Biblioteca estándar	Documentación HTML desde los docstrings	`docs/`, `docs/anexo_iv/pydoc/`
Xvfb	Paquete del sistema (Ubuntu)	Pantalla virtual para las capturas de la interfaz	`capturar_pantallas.py`
Visual Studio Code	Última versión disponible	Editor de código del equipo	Configuración no versionada (`.gitignore` excluye `.vscode/`)

3. Organización del proyecto

3.1 Estructura del repositorio

El repositorio contiene 111 archivos versionados (`git ls-files`): 54 módulos Python, 2 archivos KV, la configuración, el README y la carpeta `docs/` con la documentación pydoc. La Figura 1 muestra el árbol y la Tabla 5 describe cada archivo en una línea.

```
$ git ls-files  (árbol del repositorio Wynik-app, commit 358dcaf)

Wynik-app/
    audio/
        __init__.py
        analizador.py
        captura.py
        escucha.py
        estimador_yin.py
        guia.py
        segmentador.py
        senal.py
    nucleo/
        __init__.py
        digitacion.py
        eventos.py
        importador_musicxml.py
        instrumentos.py
        musica.py
        notacion.py
        sesion.py
    servicios/
        __init__.py
        ajustes_service.py
        almacenamiento.py
        audio_service.py
        detector_service.py
        historial_service.py
        notacion_service.py
        partitura_service.py
        practica_service.py
        progreso_service.py
        reproductor_guia.py
    tests/
        __init__.py
        test_ajustes.py
        test_almacenamiento.py
        test_musica.py
        test_progreso.py
    vistas/
        __init__.py
        app.py
        avisos.py
        componentes.kv
        componentes.py
        dialogs.py
        fuentes.py
        screens/
            __init__.py
            ajustes_inicio_screen.py
            ajustes_screen.py
            elegir_partitura_screen.py
            interprete_screen.py
            menu_screen.py
            partitura_simple_screen.py
            progreso_screen.py
        tema.py
        widgets/
            __init__.py
            dibujo.py
            guia_instrumento_widget.py
            ilustraciones.py
            partitura_simple_widget.py
            partitura_widget.py
        wynik.kv
    docs/
        49 archivos .html de pydoc + _pydoc.css
    .gitignore
    main.py
    pyproject.toml
    README.md
    requirements-escritorio.in
    requirements-escritorio.txt

111 archivos versionados (git ls-files)
```

Figura 1. Árbol del repositorio a nivel de archivo (commit 358dcaf)

Salida de `git ls-files` renderizada como imagen por `generar_evidencias.py`.

Además de los archivos versionados, la aplicación crea en la carpeta de datos (por defecto, la raíz del proyecto) `ajustes_wynik.json`, `historial_partituras.json`, `progreso_sesiones.json` y `.cache/temp_guia.wav`. Son datos locales y `.gitignore` los excluye, junto con `.venv/`, `__pycache__/` y los archivos de herramientas de desarrollo.

Tabla 5. Estructura del repositorio a nivel de archivo, con la responsabilidad de cada archivo

Archivo	Descripción
Raíz	
main.py	Punto de entrada: fija la ventana de escritorio (1040 × 720, mínimo 360 × 340), inicializa los proveedores de audio y ejecuta `WynikApp`.

Archivo	Descripción
pyproject.toml	Metadatos del proyecto, `requires-python`, dependencias mínimas, dependencias opcionales de desarrollo y configuración de pytest y Ruff.
requirements-escritorio.in	Dependencias directas fijadas (5 paquetes).
requirements-escritorio.txt	Resolución completa fijada (42 paquetes) generada con `uv pip compile`.
README.md	Descripción, funcionalidades, arquitectura e instalación de escritorio.
.gitignore	Excluye entornos, cachés, datos locales (JSON, WAV) y archivos de herramientas de desarrollo.
audio/ — captura y análisis de audio (solo NumPy; sounddevice se importa al abrir el micrófono)	
__init__.py	Descripción del paquete; no importa nada al cargarse.
captura.py	`CapturaBase`, `CapturaSounddevice` (micrófono de escritorio), `CapturaManual` (sin dispositivo), `ColaDeBloques`, `BloqueAudio` y `dispositivos_de_entrada`.
escucha.py	`MotorDeEscucha`: captura + cola + `Analizador` en un hilo propio; calibración del ruido.
analizador.py	`Analizador`: bloques → saltos de 10 ms → nivel, flujo espectral y altura YIN → eventos.
estimador_yin.py	`EstimadorYin` (YIN vectorizado), `EstimacionAltura` y `refinar_fundamental`.
segmentador.py	`Segmentador`: máquina de estados de ataques, confirmación y fin de nota.
senal.py	Nivel en dBFS, `PerfilDeRuido`, calibración y detección de interferencias (zumbido de red).
guia.py	Síntesis de la guía de audio (timbres seno, piano y guitarra) y escritura WAV con `wave`.
nucleo/ — modelo musical y reglas, sin Kivy ni audio	
__init__.py	Descripción del paquete.
musica.py	`Altura`, conversiones MIDI ↔ Hz, `comparar_frecuencia` (tolerancias en cents), `Evento`, `Parte`, `MapaTempo`, `Partitura`, `Ataque`.
eventos.py	`TipoEvento` y `EventoInterpretacion`: contrato entre el análisis de audio y la sesión.
instrumentos.py	`PerfilInstrumento` (piano, guitarra, flauta experimental), `ParametrosDeteccion` y afinaciones de guitarra.
importador_musicxml.py	`importar_musicxml` con music21: partes, voces, ligaduras, acordes, tempo y transposición de guitarra.
sesion.py	`SesionDePractica` (ventanas, alineación, estados por nota), `notas_esperadas` y `SesionProgreso`.
notacion.py	`disponer`: maquetación del pentagrama (claves, armaduras, figuras, alteraciones).
digitacion.py	Posiciones de guitarra, diagramas de acordes y teclas de piano para la guía visual.
servicios/ — fachadas y lógica de aplicación sin Kivy	
__init__.py	Importación bajo demanda (PEP 562) de las funciones públicas.
practica_service.py	`ControladorPractica`: micrófono, calibración, cuenta previa y sesión; cola de eventos entre hilos.
partitura_service.py	`cargar_partitura`, `cargar_partitura_estructurada` y `ataque_a_nota`.
audio_service.py	`generar_guia`, `generar_audio_guia`, `duracion_wav` y `formatear_tiempo`.
reproductor_guia.py	`ReproductorGuia`: estado de reproducción de la guía sobre `SoundLoader`.
detector_service.py	`DetectorDeNotas` sobre `MotorDeEscucha` y `frecuencia_a_nota`.
notacion_service.py	`nombre_para_mostrar`, `paso_diatonico` y `nota_coincide`.

Archivo	Descripción
almacenamiento.py	Directorio de datos (`WYNIK_DATOS`, Android) y lectura/escritura atómica de JSON.
ajustes_service.py	`AjustesPractica` (13 campos) con corrección de valores fuera de rango.
historial_service.py	Historial de partituras importadas y texto de detalle para «Mis partituras».
progreso_service.py	Registro y lectura de sesiones terminadas; resumen por partitura.
vistas/ — interfaz Kivy/KivyMD	
__init__.py	Exporta pantallas, widgets, `WynikApp` y `KV_PATH`.
app.py	`WynikApp` (MDApp): navegación, propiedades que muestra el KV, tick de la práctica, guía y diálogos.
wynik.kv	`ScreenManager` y las siete pantallas.
componentes.kv / componentes.py	Componentes propios: `Etiqueta`, `Boton`, `Opcion`, `Superficie`, `BarraSuperior`, `MedidorCents`, `AnilloProgreso`, `DisposicionPractica`, `Ocultable`…
dialogs.py	Diálogos modales y selector de archivos MusicXML.
avisos.py	`SnackbarWynik`: corrige la creación del snackbar de KivyMD en macOS.
fuentes.py	Registro de DejaVu Sans (símbolos musicales) y Roboto.
tema.py	Paleta, tipografía, medidas y estados visuales.
screens/ (8 archivos)	`MenuScreen`, `ElegirPartituraScreen`, `AjustesInicioScreen`, `InterpreteScreen`, `PartituraSimpleScreen`, `AjustesScreen`, `ProgresoScreen` y su `__init__.py`.
widgets/ (6 archivos)	`PartituraWidget`, `PartituraSimpleWidget`, `GuiaInstrumentoWidget`, `ilustraciones.py`, `dibujo.py` y `__init__.py`.
tests/ — pruebas unittest	
test_ajustes.py · test_almacenamiento.py · test_musica.py · test_progreso.py	13 pruebas en total (detalle en la Tabla 30).
__init__.py	Añade la raíz del repositorio a `sys.path`.
docs/	
49 archivos .html + _pydoc.css	Documentación pydoc versionada en el commit 358dcaf (48 módulos e índice).

3.2 Configuración y dependencias

La configuración se reparte en tres archivos que cumplen papeles distintos:

- `pyproject.toml` declara el proyecto (`wynik` 0.1.0), `requires-python = ">=3.13"`, las dependencias con versión mínima (`kivy>=2.3.1`, `kivymd>=2.0.0`, `numpy>=2.5.3`, `sounddevice>=0.5.6`, `music21>=10.5.0`), las dependencias opcionales `dev` (`pytest>=8.0.0`, `ruff>=0.5.0`) y la configuración de Ruff (`line-length = 100`, reglas E, F, W e I, sin E402 ni E501).
- `requirements-escritorio.in` fija con == las cinco dependencias directas.
- `requirements-escritorio.txt` es la resolución completa (42 paquetes, cinco solo para Windows mediante marcadores `sys_platform == 'win32'`), generada a partir del `.in`. Es el archivo que se instala.

```
# Comando que figura en la cabecera de requirements-escritorio.txt
uv pip compile requirements-escritorio.in -c requirements.txt --universal \
    --python-version 3.13 -o requirements-escritorio.txt
```

El archivo de restricciones `requirements.txt` que usa el comando no forma parte del repositorio; sin la opción `-c`, la resolución puede seleccionar versiones transitivas más recientes. Las pruebas se ejecutan con unittest; pytest figura solo como dependencia opcional de desarrollo.

La Tabla 6 contrasta cada dependencia directa en los cuatro lugares donde aparece.

Tabla 6. Dependencias directas: versión declarada, fijada e instalada

Paquete	pyproject.toml	.in / .txt	Instalada (macOS)	Instalada (Ubuntu)	Origen
kivy	>= 2.3.1	2.3.1	2.3.1	2.3.1	PyPI
kivymd	>= 2.0.0	2.0.0	2.0.0	2.0.0	PyPI; `kivymd._version`: release = False, fecha 2026-08-06
numpy	>= 2.5.3	2.5.3	2.5.3	2.5.3	PyPI
sounddevice	>= 0.5.6	0.5.6	0.5.6	0.5.6	PyPI
music21	>= 10.5.0	10.5.0	10.5.0	10.5.0	PyPI

En el entorno de macOS, la versión instalada se obtuvo de los metadatos `*.dist-info` (37 paquetes, sin los cinco exclusivos de Windows). KivyMD se instaló desde PyPI, como indica la ausencia de `direct_url.json`, y la versión 2.0.0 figura publicada en ese índice. librosa ya no forma parte del proyecto: según la cabecera del archivo `.in`, desde el «Ciclo 2» la detección usa una implementación propia de YIN con NumPy.

3.3 Librerías de terceros y licencias

La Tabla 7 resume las librerías directas y las transitivas más relevantes, con la licencia declarada por cada paquete instalado.

Tabla 7. Librerías de terceros y sus licencias

Librería	Versión	Licencia	Uso en Wynik
Kivy	2.3.1	MIT	Ventana, eventos, pantallas, dibujo, reloj y audio
KivyMD	2.0.0	MIT	`MDApp`, tema, fuente de iconos, Roboto y snackbar
NumPy	2.5.3	BSD-3-Clause AND 0BSD AND MIT AND Zlib AND CC0-1.0	FFT, YIN, niveles y síntesis de la guía
sounddevice	0.5.6	MIT	Captura del micrófono (PortAudio)
music21	10.5.0	BSD-3-Clause	Lectura de MusicXML
Pillow (vía KivyMD)	12.3.0	MIT-CMU	Carga de imágenes en KivyMD
materialyoucolor (vía KivyMD)	3.0.4	MIT	Colores dinámicos de KivyMD
materialshapes (vía KivyMD)	0.3	MIT	Formas de KivyMD
pycairo (vía materialshapes)	1.29.1	LGPL-2.1-only OR MPL-1.1	Dibujo de formas; se compila en macOS y Linux
asynckivy / asyncgui (vía KivyMD)	0.6.4 / 0.6.3	MIT	Tareas asíncronas de KivyMD
matplotlib (vía music21)	3.11.2	Licencia propia basada en la PSF	Dependencia de music21; Wynik no la usa
requests (vía Kivy y music21)	2.34.2	Apache-2.0	Dependencia transitiva

3.4 Código fuente por paquete

La Figura 2 muestra las dependencias entre paquetes obtenidas de las sentencias `import` del código: `nucleo/` no depende de ningún otro paquete del proyecto, `audio/` solo depende de `nucleo/`, `servicios/` une ambos y `vistas/` los usa a todos. music21 y sounddevice se importan solo cuando se necesitan (al leer un archivo o al abrir el micrófono). La Figura 3 sigue el dato desde el micrófono hasta la interfaz y el JSON, con los tres hilos que intervienen.

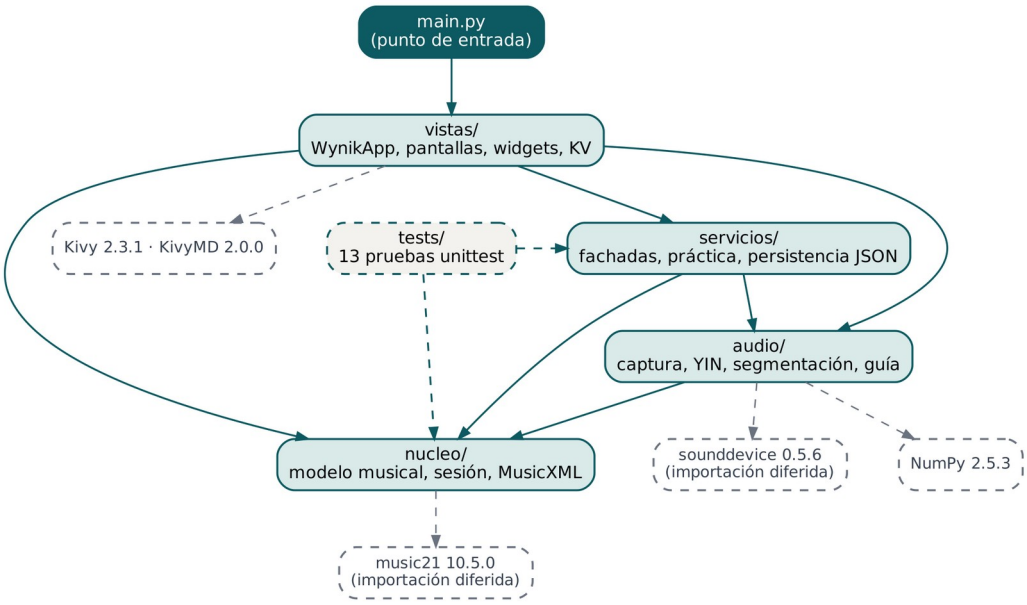


Figura 2**.** Dependencias entre paquetes y con las librerías externas

Obtenidas de las sentencias `import` del código. Flechas continuas: paquetes del proyecto; discontinuas: librerías externas.

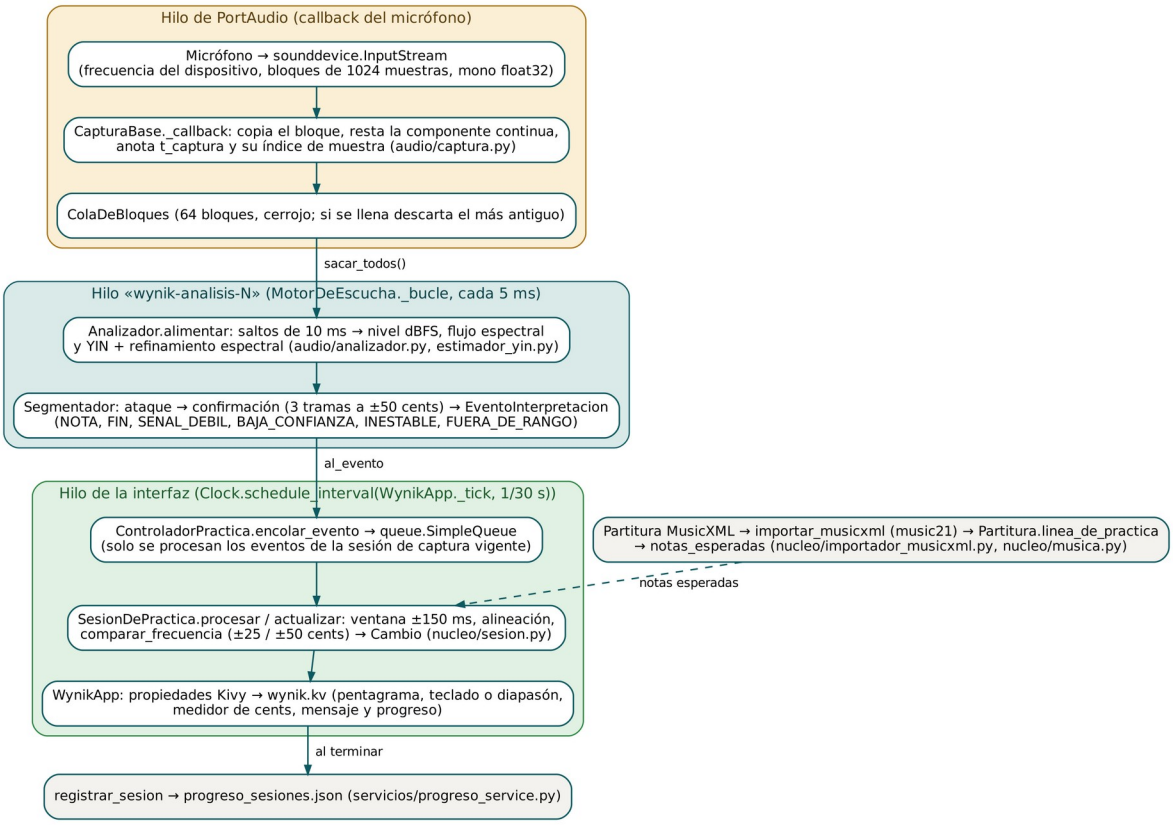


Figura 3**.** Flujo de datos: micrófono → captura → detección → comparación → puntaje → interfaz → JSON

Cada recuadro de color es un hilo de ejecución; fuente: `diagramas/flujo_datos.dot`.

Las tablas 8 a 11 describen las clases y funciones principales de cada paquete y quién las llama. La estadística de docstrings está en el apartado 5.4.

Tabla 8**.** Paquete audio/: clases y funciones principales

Elemento	Responsabilidad	Lo usa
`CapturaSounddevice` (captura.py)	Abre `sounddevice.InputStream` (mono, float32, bloques de 1024 muestras, frecuencia del dispositivo) y entrega cada bloque a la cola desde el callback.	`MotorDeEscucha`, `ControladorPractica`, `WynikApp._crear_captura`
`CapturaManual` (captura.py)	Misma interfaz sin dispositivo: los bloques llegan llamando a `_callback`.	Pruebas y simulaciones sin micrófono
`ColaDeBloques` (captura.py)	Cola acotada segura entre hilos (64 bloques); llena, descarta el bloque más antiguo.	`CapturaBase`, `MotorDeEscucha`
`MotorDeEscucha` (escucha.py)	Hilo de análisis `wynik-analisis-N`: cada 5 ms saca los bloques y los pasa al `Analizador`; calibra el ruido.	`ControladorPractica`, `WynikApp.calibrar_ahora`
`Analizador` (analizador.py)	Saltos de 10 ms: nivel, flujo espectral, YIN y refinamiento; entrega tramas al `Segmentador`.	`MotorDeEscucha`
`EstimadorYin` (estimador_yin.py)	YIN vectorizado por lotes con FFT; `refinar_fundamental` corrige la inarmonicidad del piano.	`Analizador`, `calibrar`
`Segmentador` (segmentador.py)	Máquina de estados: ataque, confirmación, diagnóstico y fin de nota → `EventoInterpretacion`.	`Analizador`
`PerfilDeRuido`, `calibrar`, `es_interferencia` (senal.py)	Niveles de silencio y mínimo, tonos estacionarios del fondo y zumbido de red.	`Analizador`, `MotorDeEscucha`
`sintetizar`, `escribir_wav` (guia.py)	Guía de audio a 22 050 Hz con timbre seno, piano o guitarra; WAV mono de 16 bits.	`servicios.audio_service`

Tabla 9. Paquete nucleo/: clases y funciones principales

Elemento	Responsabilidad	Lo usa
`Altura`, `midi_a_frecuencia`, `frecuencia_a_midi` (musica.py)	Alturas con su grafía original y conversiones MIDI ↔ Hz (La4 = 440 Hz por defecto).	Todo el proyecto
`comparar_frecuencia`, `EstadoAltura` (musica.py)	Clasifica la altura tocada: correcta, desafinada, octava incorrecta o nota incorrecta.	`SesionDePractica`
`Partitura`, `Parte`, `Evento`, `MapaTempo`, `Ataque` (musica.py)	Modelo de la partitura; `linea_de_practica` agrupa ligaduras y acordes en ataques.	Importador, servicios, vistas
`importar_musicxml` (importador_musicxml.py)	Lee MusicXML con music21 y decide la transposición de la guitarra.	`partitura_service`
`TipoEvento`, `EventoInterpretacion` (eventos.py)	Eventos NOTA, FIN, SENAL_DEBIL, BAJA_CONFIANZA, INESTABLE y FUERA_DE_RANGO con instante, frecuencia, confianza y latencia.	`Segmentador`, `SesionDePractica`, `WynikApp`
`PerfilInstrumento`, `ParametrosDeteccion` (instrumentos.py)	Rango, transposición, afinaciones y parámetros del detector por instrumento.	`Analizador`, `WynikApp`
`SesionDePractica`, `ConfiguracionSesion`, `EstadoNota` (sesion.py)	Compara eventos con la partitura en el tiempo y produce un `Cambio` por nota; `resumen` calcula la precisión.	`ControladorPractica`
`SesionProgreso` (sesion.py)	Resultado de una sesión con `to_dict` / `from_dict` para el JSON.	`progreso_service`
`disponer` (notacion.py) · digitacion.py	Maquetación del pentagrama; posiciones de guitarra y teclas de piano.	`PartituraWidget`, `GuiaInstrumentoWidget`

Tabla 10**.** Paquete servicios/: clases y funciones principales

Elemento	Responsabilidad	Lo usa
`ControladorPractica` (practica_service.py)	Secuencia Empezar → calibración (opcional, 1 s) → cuenta previa → sesión; pausa y reanudación; cola de eventos del hilo de análisis.	`WynikApp.iniciar_escucha`, `_tick`
`cargar_partitura_estructurada`, `ataque_a_nota` (partitura_service.py)	Fachada del importador y conversión de cada ataque al diccionario que usa la interfaz.	`WynikApp._cargar_partitura`
`generar_guia` (audio_service.py) · `ReproductorGuia` (reproductor_guia.py)	Genera la guía WAV con los tiempos de la partitura y gestiona su reproducción.	`WynikApp.preparar_audio_guia`
`leer_json`, `escribir_json`, `configurar_directorio` (almacenamiento.py)	Ubicación de los datos y escritura atómica (archivo temporal + `os.replace`).	Servicios de ajustes y progreso
`AjustesPractica`, `cargar_ajustes`, `guardar_ajustes` (ajustes_service.py)	Ajustes persistentes con corrección de valores fuera de rango.	`WynikApp._cargar_ajustes`, `_guardar_ajustes`
`cargar_historial`, `agregar_al_historial` (historial_service.py)	Historial de partituras importadas.	`WynikApp`
`registrar_sesion`, `cargar_progreso`, `resumen_por_partitura` (progreso_service.py)	Sesiones terminadas (máximo 500) y resumen por partitura.	`WynikApp._terminar`, `refrescar_progreso`

Tabla 11**.** Paquete vistas/: clases principales

Elemento	Responsabilidad	Lo usa
`WynikApp` (app.py)	Carga los KV una sola vez en `build()`, traduce el estado de la práctica a propiedades Kivy, navega entre pantallas y ejecuta `_tick` a 30 Hz.	`main.py`
Pantallas (screens/)	Siete subclases de `Screen` con poca lógica (`on_enter`, `on_leave`); el diseño está en `wynik.kv`.	`ScreenManager` de `wynik.kv`
`PartituraWidget`, `PartituraSimpleWidget` (widgets/)	Dibujan el pentagrama o la cinta de notas con el estado de cada nota.	`InterpreteScreen`, `PartituraSimpleScreen`
`GuiaInstrumentoWidget` (widgets/)	Teclado o diapasón con la nota esperada y la detectada.	Pantallas de práctica
Componentes (componentes.py / .kv)	Controles propios: `Boton`, `Opcion`, `GrupoOpciones`, `Superficie`, `BarraSuperior`, `Chip`, `EstadoPildora`, `MedidorCents`, `TiraResultados`, `AnilloProgreso`, `GraficoSesiones`, `DisposicionPractica`.	`wynik.kv`, `app.py`, `dialogs.py`
`mostrar_dialogo`, `abrir_selector_de_archivo` (dialogs.py)	Diálogos modales y selector `FileChooserListView` con filtro MusicXML.	`WynikApp`

3.5 Archivos KV

La interfaz se declara en dos archivos KV que `WynikApp.build()` carga una sola vez; `load_kv` se anula a propósito porque Kivy cargaba `wynik.kv` por el nombre de la clase y cada regla se aplicaba dos veces. La Tabla 12 resume su contenido.

Tabla 12**.** Archivos KV y reglas que definen

Archivo	Contenido	Reglas principales
`vistas/wynik.kv` (1079 líneas)	Raíz `ScreenManager` con transición `SlideTransition` (0,22 s), fondo común de `Screen` y la disposición de las siete pantallas.	`<MenuScreen>` (menu), `<ElegirPartituraScreen>` (elegir_partitura), `<AjustesInicioScreen>` (ajustes_inicio), `<InterpreteScreen>` (interprete), `<PartituraSimpleScreen>` (partitura_simple), `<AjustesScreen>` (ajustes), `<ProgresoScreen>` (progreso)
`vistas/componentes.kv` (505 líneas)	27 reglas de componentes reutilizables con los colores y medidas de `tema.py`.	`Etiqueta`, `Rotulo`, `Seccion`, `Secundario`, `Icono`, `Boton`, `BotonIcono`, `Superficie`, `Tarjeta`, `TarjetaAjuste`, `Opcion`, `GrupoOpciones`, `Paso`, `Chip`, `EstadoPildora`, `BarraSuperior`, `BarraAccion`, `Mosaico`, `FilaPartitura`, `FilaSesion`, `FilaPartituraProgreso`…

3.6 Algoritmo de detección de notas

La detección es monofónica (una nota a la vez) y se divide en cinco etapas. El código cita como base el estimador YIN (de Cheveigné y Kawahara, 2002) y, para los ataques, una versión simplificada de SuperFlux (Böck y Widmer, 2013). Ambas están implementadas con NumPy, sin librosa.

3.6.1 Captura y paso entre hilos

`CapturaSounddevice` abre el micrófono con la **frecuencia de muestreo del dispositivo** (`default_samplerate`; habitualmente 44 100 o 48 000 Hz, no una constante del código), un canal, `float32` y **bloques de 1024 muestras** (unos 21 ms a 48 kHz). El callback de PortAudio solo copia el bloque, resta la componente continua, anota el instante y el número de su primera muestra y lo deja en `ColaDeBloques`. Ningún cálculo de altura se hace dentro del callback.

El hilo de análisis de `MotorDeEscucha` vacía la cola cada 5 ms y produce `EventoInterpretacion`. Cada evento llega por `ControladorPractica.encolar_evento` a una `queue.SimpleQueue`. El hilo de la interfaz ejecuta `WynikApp._tick` cada 1/30 s con `Clock.schedule_interval`; ahí `ControladorPractica.actualizar()` vacía la cola, pasa cada evento a `SesionDePractica` y aplica los cambios a las propiedades Kivy que observa el KV. Así, ningún objeto de Kivy se toca fuera del hilo de la interfaz (Figura 3).

3.6.2 Estimación de altura (YIN)

Por cada salto de 10 ms, `Analizador` toma la trama que termina en ese salto y `EstimadorYin` calcula: (1) la función de diferencia d(τ) mediante correlación cruzada con FFT y sumas acumuladas; (2) la diferencia media acumulada normalizada d′(τ); (3) el primer mínimo de d′(τ) bajo el umbral absoluto (0,30 en los perfiles); (4) la interpolación parabólica del mínimo; y (5) una comprobación de subarmónicos por debajo de 1 kHz que evita errores de octava en notas

graves. La aperiodicidad d′(τ) da la confianza (1 − d′). En el piano, `refinar_fundamental` busca el pico espectral real entre −60 y +15 cents de la estimación (ventana de Hann, FFT de 65 536 puntos), porque la inarmonicidad de las cuerdas eleva unos 16 cents la estimación de YIN.

El rango de búsqueda es el del instrumento con un margen: piano de A0 − 2 semitonos a C8 + 4 semitonos (24,50–5274,0 Hz) y guitarra de E2 − 12 a E6 + 12 semitonos (41,20–2637,0 Hz), limitado al 45 % de la frecuencia de muestreo. La trama necesaria crece con la nota más grave: 81,7 ms en piano y 48,6 ms en guitarra (Tabla 14).

3.6.3 Segmentación en eventos

`Segmentador` convierte las tramas en eventos. Un ataque se detecta por **energía** (el nivel sube 6 dB sobre el mínimo de los 50 ms anteriores) o por **flujo espectral** (ventana de Hann de 20 ms, compresión log10(1 + 1000·|X|), filtro de máximos de 3 bandas), y también hay candidatos sin ataque cuando aparece una altura estable o la altura cambia al menos 0,75 semitonos (legato). Un candidato se **confirma** con 3 tramas periódicas seguidas dentro de ±50 cents de su mediana; si no se confirma en 150 ms (más la trama) se informa como señal débil, baja confianza o altura inestable. La nota termina tras 40 ms de silencio o 120 ms sin periodicidad (evento FIN). Los tonos estables a ±20 cents de 50/60 Hz y sus armónicos, sin ataque, se tratan como zumbido de red.

3.6.4 Conversión de frecuencia a nota y tolerancia en cents

La frecuencia confirmada se convierte a número MIDI fraccionario con m = 69 + 12·log2(f / La4), donde La4 es 440 Hz por defecto y se ajusta entre 430 y 450 Hz. La nota más cercana es ⌊m + 0,5⌋ y la desviación, (m − nota)·100 cents. `comparar_frecuencia` aplica las tolerancias de la Tabla 13; la precisión de la sesión es aciertos / notas evaluables, donde un acierto es una nota correcta o desafinada y los acordes no se evalúan.

Tabla 13**.** Clasificación de una nota detectada frente a la esperada (`nucleo/musica.py`)

Desviación respecto de la nota esperada	Estado	¿Cuenta como acierto?
|desviación| ≤ 25 cents	CORRECTA (afinada)	Sí
25 < |desviación| ≤ 50 cents	DESAFINADA (misma nota y octava)	Sí
> 50 cents y misma clase de altura	OCTAVA_INCORRECTA	No
> 50 cents y otra clase de altura	NOTA_INCORRECTA	No
Sin ataque en la ventana / solo diagnósticos	OMITIDA, SENAL_INSUFICIENTE, BAJA_CONFIANZA o FUERA_DE_RANGO	No
Acorde (detector monofónico)	SIN_EVALUAR	No se cuenta

3.6.5 Sincronización con el tempo

`notas_esperadas` toma el inicio y la duración de cada ataque en segundos según el tempo de la partitura (con sus cambios) y los divide por el factor de tempo elegido (50–150 %). En el modo **Seguir el tempo**, cada nota tiene una ventana de ±150 ms (configurable a 100, 200 o 250 ms) alrededor de su inicio esperado; la primera nota admite hasta 500 ms de retraso y el primer acierto fija el desfase entre el reloj y la partitura. Si los tres últimos aciertos llegan desplazados en el mismo sentido, el desfase se corrige con su media. La ventana se cierra

0,35 s después de su fin (`MARGEN_CIERRE_S`) para no perder ataques ya tocados que aún se están confirmando. En el modo **Esperar cada nota**, la partitura avanza solo con un acierto. Antes de empezar hay una cuenta previa visual (3 pulsos por defecto, de 60/tempo segundos acotados a 0,3–1,2 s).

3.6.6 Parámetros efectivos

Los valores de la Tabla 14 se obtuvieron instanciando las clases del código (`generar_evidencias.py`). Según el docstring de `ParametrosDeteccion`, corresponden a valores iniciales ajustados con señales sintéticas; la prueba con piano real se describe en el apartado 5.8.1.

Tabla 14**.** Parámetros del detector calculados con las clases del código (`evidencias/parametros_deteccion.txt`)

Parámetro	Piano	Guitarra	Origen en el código
Frecuencia de muestreo	La del dispositivo (44 100 / 48 000 Hz)	Ídem	`CapturaSounddevice.abrir`
Bloque de captura · cola	1024 muestras · 64 bloques	Ídem	`TAM_BLOQUE`, `CAPACIDAD_COLA_BLOQUES`
Salto de análisis	10 ms (480 muestras a 48 kHz)	Ídem	`DURACION_SALTO_S`
Rango de búsqueda	24,50–5274,0 Hz	41,20–2637,0 Hz	`PerfilInstrumento.rango_busqueda_hz`
Trama YIN a 48 kHz	3923 muestras (81,7 ms)	2333 muestras (48,6 ms)	`EstimadorYin.longitud_trama`
Umbral YIN · aperiodicidad máxima	0,30 · 0,30	0,30 · 0,30	`ParametrosDeteccion`
Refinamiento espectral	Sí	No	`refinar_fundamental`
Confirmación	3 tramas a ±50 cents; máximo 150 ms	Ídem	`tramas_confirmacion`, `confirmacion_max_s`
Latencia mínima de confirmación	111,7 ms	78,6 ms	`Analizador.latencia_minima_s`
Niveles sin calibrar	Silencio −65 dBFS · mínimo −50 dBFS	Ídem	`nivel_silencio_dbfs`, `nivel_minimo_dbfs`
Tolerancias de altura	±25 cents afinada · ±50 cents misma nota	Ídem	`TOLERANCIA_*_CENTS`
Ventana temporal · cierre	±150 ms · +0,35 s	Ídem	`ConfiguracionSesion`

3.6.7 Verificación con señal sintética

Para comprobar la cadena completa sin micrófono se generó una escala de Do mayor a negra = 80 con ondas senoidales a 48 kHz y tres errores a propósito: Fa4 en lugar de Re4, Fa4 desafinado +35 cents y Si4 omitido. La señal se analizó con `Analizador` (perfil piano) y los eventos se evaluaron con `SesionDePractica`. La Tabla 15 muestra el resultado: los ocho estados coinciden con lo esperado, la latencia de confirmación fue de 100 ms por nota y la precisión, 6/8 (75 %). La misma secuencia se reprodujo en la interfaz de la aplicación (figuras 14 a 17).

Tabla 15. Resultado de la cadena de detección con entrada simulada (escala de Do mayor, piano, 48 kHz)

Nota esperada	Señal simulada	Evento detectado	Estado asignado
C4	C4 afinado	NOTA C4, 0 cents, confianza 0,994	CORRECTA
D4	F4 (nota equivocada)	NOTA F4, 0 cents	NOTA_INCORRECTA
E4	E4 afinado	NOTA E4, 0 cents	CORRECTA
F4	F4 + 35 cents	NOTA F4, +35 cents	DESAFINADA
G4 · A4	Afinadas	NOTA G4 · NOTA A4	CORRECTA · CORRECTA
B4	Silencio	—	OMITIDA
C5	C5 afinado	NOTA C5, 0 cents	CORRECTA

3.7 Persistencia JSON

No hay base de datos, servicios remotos ni inicio de sesión. Todos los datos se guardan en archivos JSON locales dentro del **directorio de datos**, que se resuelve en este orden (`servicios/almacenamiento.py`): (1) el que fija `configurar_directorio()`, que en Android es `app.user_data_dir`; (2) la variable de entorno `WYNIK_DATOS`; y (3) por defecto, la raíz del proyecto. La guía de audio se escribe en `.cache/temp_guia.wav` dentro de ese directorio. La Tabla 16 detalla cada archivo.

Tabla 16. Archivos de datos locales

Archivo	Contenido	Cuándo se escribe	Módulo · escritura
`ajustes_wynik.json`	Objeto con los 13 campos de `AjustesPractica` (instrumento, afinación, La4, modo, factor de tempo, tolerancia, notación, calibración, pulsos, dispositivo, volumen, vista, escala de fuente).	Cada vez que cambia un ajuste (`WynikApp._ajuste_cambiado` → `_guardar_ajustes`).	`ajustes_service` · atómica
`historial_partituras.json`	Lista de objetos `{nombre, ruta}` con la ruta absoluta de cada partitura.	Al importar una partitura que no estaba (`agregar_al_historial`).	`historial_service` · directa (no atómica)
`progreso_sesiones.json`	Lista de sesiones (`SesionProgreso.to_dict`), máximo 500.	Al terminar una práctica (`WynikApp._terminar` → `registrar_sesion`).	`progreso_service` · atómica
`.cache/temp_guia.wav`	Guía de audio (WAV mono, 16 bits, 22 050 Hz).	Al preparar la práctica o cambiar un ajuste que afecta la guía.	`audio_service` · `wave`

Los ejemplos siguientes fueron escritos por la aplicación durante la sesión de captura del capítulo 4, con `WYNIK_DATOS=docs/anexo_iv/_datos_demo`; en las rutas absolutas, la carpeta del repositorio se abrevia como `<raíz>`.

progreso_sesiones.json (una sesión con un error de nota, una nota desafinada y una omitida):


```
[
  {
    "fecha": "2026-11-04T15:30:59-03:00",
    "partitura": "escala_do_mayor",
    "instrumento": "piano",
    "modo": "seguir_tempo",
    "factor_tempo": 1.0,
    "total": 8,
    "aciertos": 6,
    "precision": 0.75,
    "estados": {
      "pendiente": 0,
      "correcta": 5,
      "desafinada": 1,
      "nota_incorrecta": 1,
      "octava_incorrecta": 0,
      "omitida": 1,
      "senal_insuficiente": 0,
      "baja_confianza": 0,
      "fuera_de_rango": 0,
      "sin_evaluar": 0
    },
    "cents_medios": 5.841643747879388,
    "desfase_medio_s": 0.01905938822221742
  }
]
```


historial_partituras.json:


```
[
  {
    "nombre": "escala_do_mayor",
    "ruta": "<raíz>/docs/anexo_iv/partituras/escala_do_mayor.musicxml"
  },
  {
    "nombre": "oda_a_la_alegria_guitarra",
    "ruta": "<raíz>/docs/anexo_iv/partituras/oda_a_la_alegria_guitarra.musicxml"
  }
]
```

ajustes_wynik.json:

```
{
  "instrumento": "piano",
  "afinacion_guitarra": "estandar",
  "la4": 440.0,
  "modo": "seguir_tempo",
  "factor_tempo": 1.0,
  "tolerancia_ms": 150,
  "notacion_solfeo": false,
  "calibrar_al_empezar": false,
  "pulsos_cuenta": 3,
  "dispositivo": null,
  "volumen_guia": 0.8,
  "vista_simple": false,
  "escala_fuente": 1.0
}
```

Durante la lectura, un archivo ilegible se trata como vacío y se informa por consola (`[Wynik] No se pudo leer …`), y los ajustes fuera de rango se corrigen (`AjustesPractica.corregidos`). El historial guarda rutas absolutas, por lo que es propio de cada computador.

3.8 Recursos gráficos

El repositorio no contiene imágenes de mapa de bits. Los gráficos se dibujan con instrucciones del canvas de Kivy: las ilustraciones de instrumentos y la portada (`widgets/ilustraciones.py`, `widgets/dibujo.py`), el pentagrama (`widgets/partitura_widget.py`), la cinta simplificada y la guía de teclado o diapasón (`widgets/guia_instrumento_widget.py`). Los iconos son glifos de la fuente «Icons» de KivyMD. `vistas/fuentes.py` registra DejaVu Sans (incluida en Kivy) como fuente de texto porque contiene ♭, ♯, ♮, ✓ y ✗, que Roboto no tiene, y Roboto Medium y Bold para los rótulos. La Tabla 17 resume la paleta de `vistas/tema.py`.

Tabla 17. Paleta principal de la interfaz (`vistas/tema.py`)

Constante	Color	Uso
`FONDO`	#F4F2EE	Fondo de todas las pantallas
`PRIMARIO`	#0E5A62 (azul petróleo)	Barras, botones principales, nota esperada
`CORRECTO`	#2A8641 (verde)	Nota correcta (✓)
`DESAFINADO`	#A87110 (ocre)	Nota desafinada (≈)
`ERROR`	#C43A2D (rojo)	Nota u octava equivocada (×), fuera de rango
`NEUTRO`	RGB (0,42; 0,45; 0,48)	Pendiente, sin tocar, sin evaluar

4. Interfaz de la aplicación

Las capturas de este capítulo se obtuvieron con el script `docs/anexo_iv/capturar_pantallas.py`, que importa `vistas.WynikApp` (la misma clase que ejecuta `main.py`), recorre las pantallas del `ScreenManager` con `Clock` y guarda cada una con `Window.screenshot()`. Todas usan el mismo tamaño de ventana, con proporción de teléfono: **412 × 915 dp con densidad 2 (824 × 1830 px)**. Se tomaron el 04/11/2026 en Linux x86_64 (Ubuntu 24.04) con la pantalla virtual Xvfb, usando partituras de ejemplo creadas con music21 (`docs/anexo_iv/partituras/`).

Para las capturas de la práctica en curso se utilizó **entrada simulada**: el script entrega a la aplicación ondas senoidales con la frecuencia de cada nota mediante `audio.captura.CapturaManual` y el atributo `WynikApp.fabrica_captura`, con la misma cadencia que un micrófono (bloques de 1024 muestras a 48 kHz). El análisis, la sesión y la interfaz corresponden al código de la aplicación, sin modificaciones.

Los marcadores numerados se dibujaron con Pillow (`anotar_capturas.py`) en la posición de cada widget, leída del árbol de widgets en el momento de la captura. Las originales están en `capturas/originales/` y las anotadas, en `capturas/anotadas/`. La Figura 4 resume la navegación.

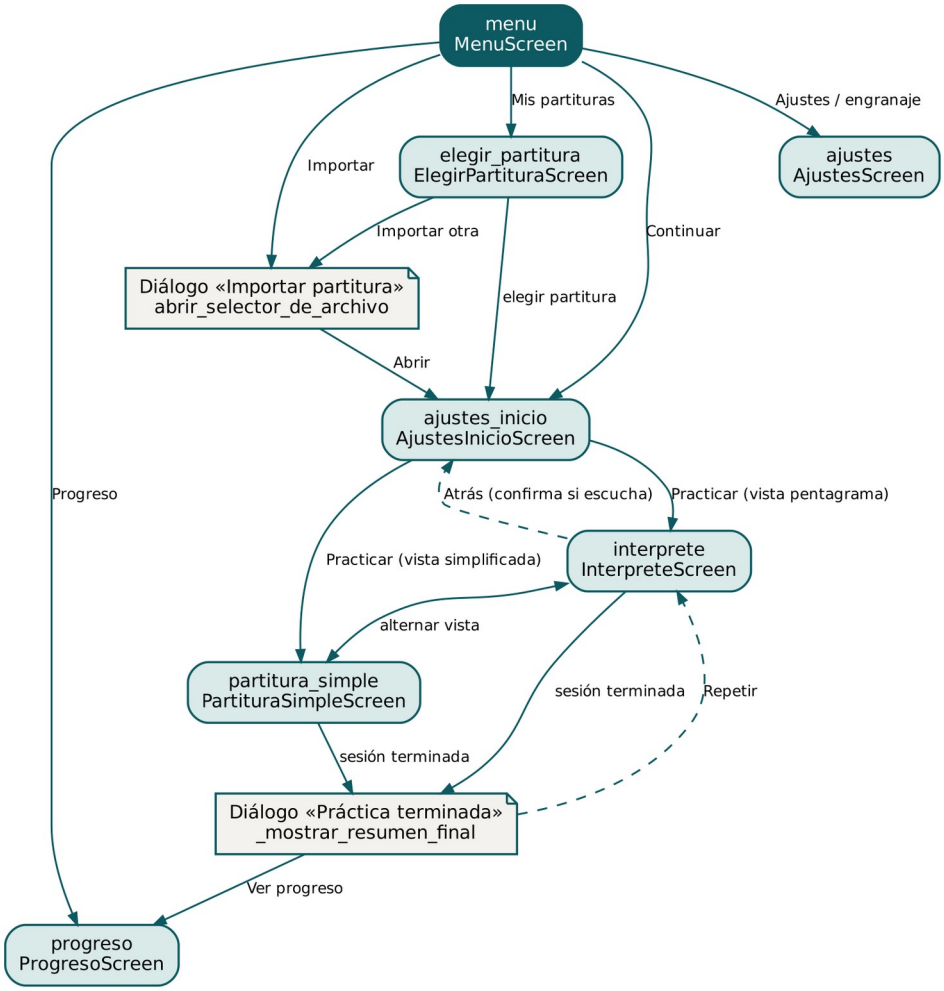


Figura 4. Navegación entre pantallas y diálogos

Las siete pantallas del `ScreenManager` de `wynik.kv` y los dos diálogos; líneas discontinuas: retorno.

4.1 Menú principal

La Figura 5 muestra la pantalla de inicio (`menu`) y la Tabla 18 describe sus elementos.
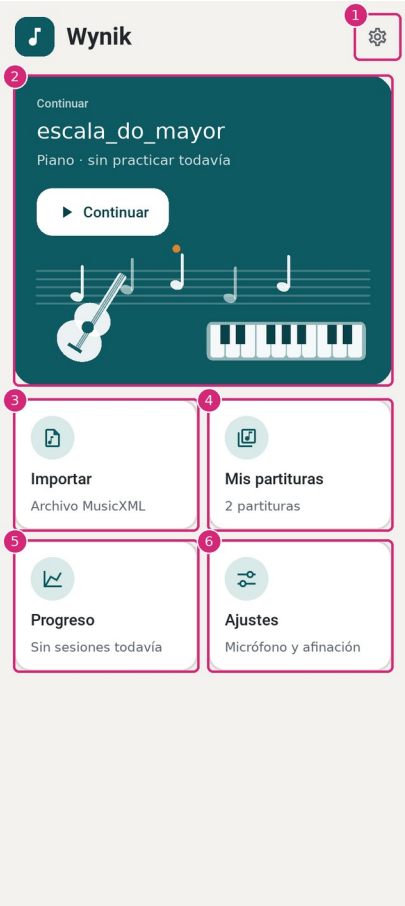


Figura 5. Menú principal con la última partitura importada

Captura de la aplicación: ventana de 412 × 915 dp (densidad 2), Linux x86_64, 04/11/2026; marcadores numerados añadidos con Pillow.

Tabla 18. Elementos del menú principal

N°	Elemento	Qué hace	Archivo / clase
1	Ajustes generales	Abre la pantalla de ajustes.	`wynik.kv` <MenuScreen> · `BotonIcono` → `WynikApp.change_screen`
2	Continuar	Muestra la última partitura y su progreso; «Continuar» la vuelve a cargar y abre la preparación.	`portada_tarjeta` · `WynikApp.practicar_ultima` · `IlustracionPortada`
3	Importar	Abre el selector de archivos MusicXML.	`Mosaico` → `WynikApp.subir_partitura`
4	Mis partituras	Abre la biblioteca; el detalle indica cuántas partituras hay.	`Mosaico` → pantalla `elegir_partitura`
5	Progreso	Abre el historial de sesiones; el detalle resume el avance.	`Mosaico` → pantalla `progreso`
6	Ajustes	Acceso alternativo a la pantalla de ajustes.	`Mosaico` → pantalla `ajustes`

4.2 Importación de partitura

El diálogo de importación (Figura 6, Tabla 19) se abre desde el menú o desde «Mis partituras».



Figura 6. Diálogo «Importar partitura» con el archivo de ejemplo seleccionado

Captura de la aplicación: ventana de 412 × 915 dp (densidad 2), Linux x86_64, 04/11/2026; marcadores numerados añadidos con Pillow.

Tabla 19. Elementos del diálogo de importación

N°	Elemento	Qué hace	Archivo / clase
1	Título e instrucción	Indica los formatos admitidos: .musicxml, .xml y .mxl.	`dialogs.mostrar_dialogo`
2	Selector de archivos	Lista la carpeta personal (~) filtrada por extensión.	`FileChooserListView` en `dialogs.abrir_selector_de_archivo`
3	Archivo seleccionado	Partitura elegida; un doble toque también la abre.	`FileChooserListView.selection` / `on_submit`
4	Cancelar	Cierra el diálogo sin cargar nada.	`mostrar_dialogo`
5	Abrir	Lee la partitura con music21, la añade al historial JSON y abre «Preparar la práctica».	`WynikApp._partitura_seleccionada` → `_cargar_partitura` → `agregar_al_historial`

4.3 Selección de partitura (Mis partituras)

La Figura 7 muestra la biblioteca construida a partir de `historial_partituras.json` (Tabla 20).

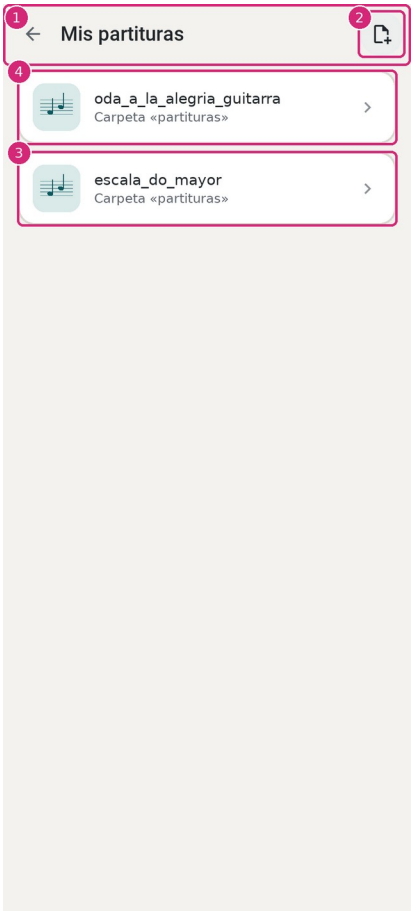


Figura 7**.** Pantalla «Mis partituras» con las dos partituras importadas

Captura de la aplicación: ventana de 412 × 915 dp (densidad 2), Linux x86_64, 04/11/2026; marcadores numerados añadidos con Pillow.

Tabla 20**.** Elementos de «Mis partituras»

N°	Elemento	Qué hace	Archivo / clase
1	Barra superior	Título y botón Atrás (vuelve al menú).	`componentes.BarraSuperior` · `WynikApp.atras`
2	Importar otra partitura	Abre el selector de archivos.	`BotonIcono` → `subir_partitura`
3	Partitura del historial	Nombre, carpeta y, si ya se practicó, sesiones y mejor precisión; al tocarla se carga y se abre la preparación.	`componentes.FilaPartitura` · `_refrescar_lista_partituras` · `detalle_de_entrada`
4	Segunda partitura	Ídem (partitura de guitarra).	Ídem

4.4 Preparación de la práctica (instrumento, modo y tempo)

La pantalla `ajustes_inicio` reúne los ajustes de la práctica de la partitura elegida. Se muestra en dos capturas: la parte superior (Figura 8, Tabla 21) y la parte inferior, con el modo y el tempo (Figura 9, Tabla 22).



Figura 8. «Preparar la práctica», parte superior

Captura de la aplicación: ventana de 412 × 915 dp (densidad 2), Linux x86_64, 04/11/2026; marcadores numerados añadidos con Pillow.

Tabla 21. Elementos de «Preparar la práctica» (parte superior)

N°	Elemento	Qué hace	Archivo / clase
1	Partitura elegida	Nombre del archivo cargado.	`ids.titulo_partitura` en <AjustesInicioScreen>
2	Datos de la partitura	Chips con número de notas, compases, tempo (♩) y parte.	`WynikApp._mostrar_datos_partitura` · `componentes.Chip`
3	Escuchar la referencia	Reproduce la guía sintetizada y muestra su duración; mientras suena, el micrófono no escucha.	`toggle_reproduccion_audio`, `preparar_audio_guia` · `audio.guia` · `ReproductorGuia`
4	Instrumento	Guitarra (E2–E6) o piano (A0–C8): fija el perfil y los parámetros de detección.	`WynikApp.cambiar_instrumento` · `nucleo.instrumentos.PERFILES`
5	Practicar	Abre la práctica con la vista elegida.	`WynikApp.preparar_practica`

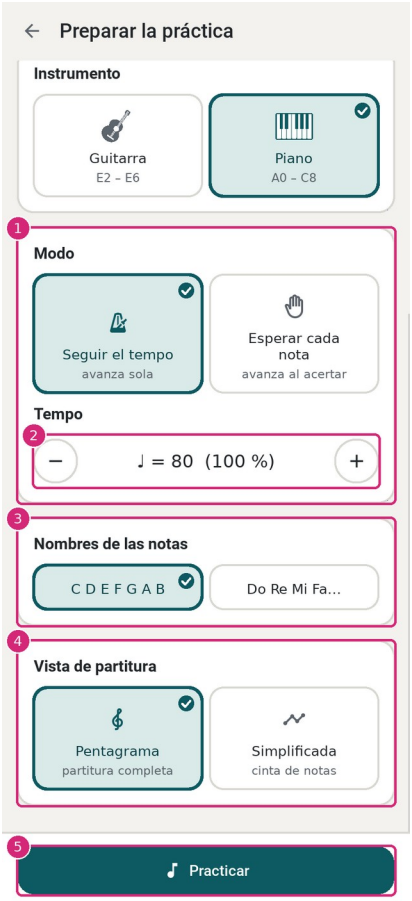


Figura 9**.** «Preparar la práctica», parte inferior: modo, tempo, notación y vista

Captura de la aplicación: ventana de 412 × 915 dp (densidad 2), Linux x86_64, 04/11/2026; marcadores numerados añadidos con Pillow.

Tabla 22**.** Elementos de «Preparar la práctica» (parte inferior)

N°	Elemento	Qué hace	Archivo / clase
1	Modo	«Seguir el tempo» (la partitura avanza sola) o «Esperar cada nota» (avanza al acertar).	`WynikApp.cambiar_modo` · `nucleo.sesion.ModoPractica`
2	Tempo	Factor de 50 % a 150 % sobre el tempo de la partitura, en pasos de 10 %.	`WynikApp.cambiar_factor_tempo` · regla `Paso`
3	Nombres de las notas	Notación anglosajona (C D E…) o solfeo (Do Re Mi…).	`WynikApp.cambiar_notacion` · `nombre_para_mostrar`
4	Vista de partitura	Pentagrama completo o cinta simplificada.	`WynikApp.cambiar_vista_simple`
5	Practicar	Ídem.	`preparar_practica`

4.5 Ajustes generales

La pantalla `ajustes` contiene los ajustes persistentes que afectan al detector y a la evaluación (Figura 10 y Figura 11; tablas 23 y 24). Cada cambio se guarda de inmediato en `ajustes_wynik.json`.



Figura 10**.** Ajustes: micrófono, ruido de fondo y afinación

Captura de la aplicación: ventana de 412 × 915 dp (densidad 2), Linux x86_64, 04/11/2026; marcadores numerados añadidos con Pillow. El entorno de captura no tiene micrófono; por eso se lee «No se encontró ningún micrófono».

Tabla 23**.** Elementos de «Ajustes» (parte superior)

N°	Elemento	Qué hace	Archivo / clase
1	Micrófono	Muestra el dispositivo de entrada; «Cambiar micrófono» recorre los disponibles.	`refrescar_dispositivo`, `siguiente_dispositivo` · `audio.captura.dispositivos_de_entrada`
2	Ruido de fondo	Mide 1 s de silencio («Medir ahora») y decide si se mide al empezar o se usa la última medida.	`calibrar_ahora`, `cambiar_calibracion` · `audio.senal.calibrar`
3	Afinación de referencia	La4 entre 430 y 450 Hz; recuerda las tolerancias de ±25 y ±50 cents.	`WynikApp.cambiar_la4` · `LA4_MIN`, `LA4_MAX`
4	Afinación de la guitarra	Estándar, drop D, medio tono abajo o re abierta: cambia el rango de la guitarra.	`cambiar_afinacion` · `AFINACIONES_GUITARRA`

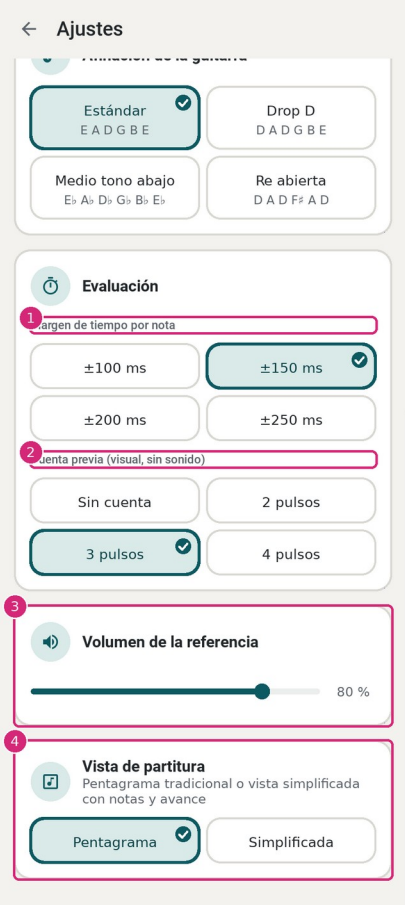


Figura 11**.** Ajustes: evaluación, volumen y vista

Captura de la aplicación: ventana de 412 × 915 dp (densidad 2), Linux x86_64, 04/11/2026; marcadores numerados añadidos con Pillow.

Tabla 24**.** Elementos de «Ajustes» (parte inferior)

N°	Elemento	Qué hace	Archivo / clase
1	Margen de tiempo por nota	Tolerancia de la ventana: ±100, ±150, ±200 o ±250 ms.	`cambiar_tolerancia` · `ConfiguracionSesion.tolerancia_s`
2	Cuenta previa	Sin cuenta, 2, 3 o 4 pulsos visuales, sin sonido para que el micrófono no los capte.	`cambiar_pulsos` · `OpcionesPractica.pulsos_cuenta`
3	Volumen de la referencia	Volumen de la guía de audio (0–100 %).	`WynikApp.ajustar_volumen`
4	Vista de partitura	Ídem que en la preparación.	`cambiar_vista_simple`

4.6 Pantalla de práctica

La Figura 12 muestra la pantalla `interprete` con la partitura cargada, antes de pulsar «Empezar». La Tabla 25 describe sus elementos; los mismos números se usan en las figuras 13 a 16.

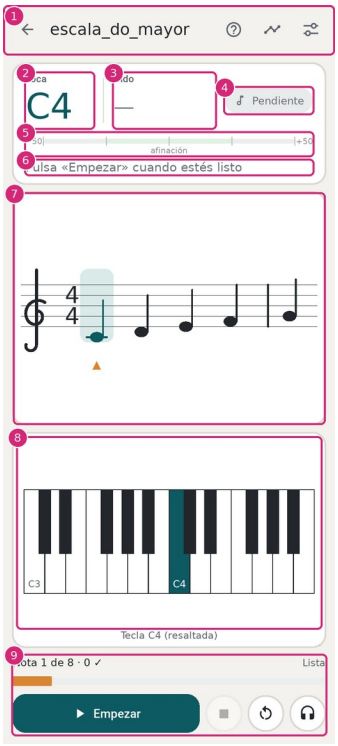


Figura 12**.** Práctica preparada (estado «Lista»)

Captura de la aplicación: ventana de 412 × 915 dp (densidad 2), Linux x86_64, 04/11/2026; marcadores numerados añadidos con Pillow.

Tabla 25**.** Elementos de la pantalla de práctica

N°	Elemento	Qué hace	Archivo / clase
1	Barra superior	Título de la partitura; leyenda de marcas (?), alternar vista y volver a la preparación.	`BarraSuperior` · `mostrar_leyenda`, `alternar_vista_practica`, `atras`
2	Nota esperada («Toca»)	Nota que se debe tocar ahora, en la notación elegida.	`WynikApp.texto_esperada` · `_mostrar_nota`
3	Nota oída («Oído»)	Última nota detectada, coloreada según el resultado.	`WynikApp.nota_detectada` · `_procesar_evento`
4	Estado de la nota	Pendiente, Prepárate, Correcta, Desafinada, Error, Sin tocar…	`componentes.EstadoPildora` · `tema.ESTADOS`
5	Medidor de afinación	Desviación de −50 a +50 cents con la zona afinada (±25).	`componentes.MedidorCents` · `cents_detectados`
6	Mensaje de resultado	Explica el resultado: «✓ C4: correcta», «× nota equivocada: tocaste F4, se esperaba D4»…	`WynikApp._aplicar_cambio`
7	Pentagrama	Partitura con la nota actual resaltada y una marca por nota (✓ ≈ × –).	`widgets.PartituraWidget` · `nucleo.notacion.disponer`
8	Guía del instrumento	Teclado (piano) o diapasón (guitarra) con la tecla esperada y la detectada.	`widgets.GuiaInstrumentoWidget` · `nucleo.digitacion`
9	Progreso y controles	«Nota n de N · aciertos», tira de resultados, estado de la escucha y botones Empezar/Pausa, detener, reiniciar y guía.	`ids.pie` · `TiraResultados` · `boton_principal`, `detener_escucha`, `reiniciar_practica`

4.7 Práctica en curso: nota correcta, incorrecta y desafinada

Al pulsar «Empezar» empieza la cuenta previa (Figura 13) y, al terminar, la escucha. Las figuras 14 a 16 muestran tres momentos de la misma sesión con entrada simulada: la primera nota tocada correctamente, un Fa4 tocado cuando se esperaba Re4 (nota incorrecta) y un Fa4 desafinado +35 cents, que cuenta como acierto pero se marca en ocre.

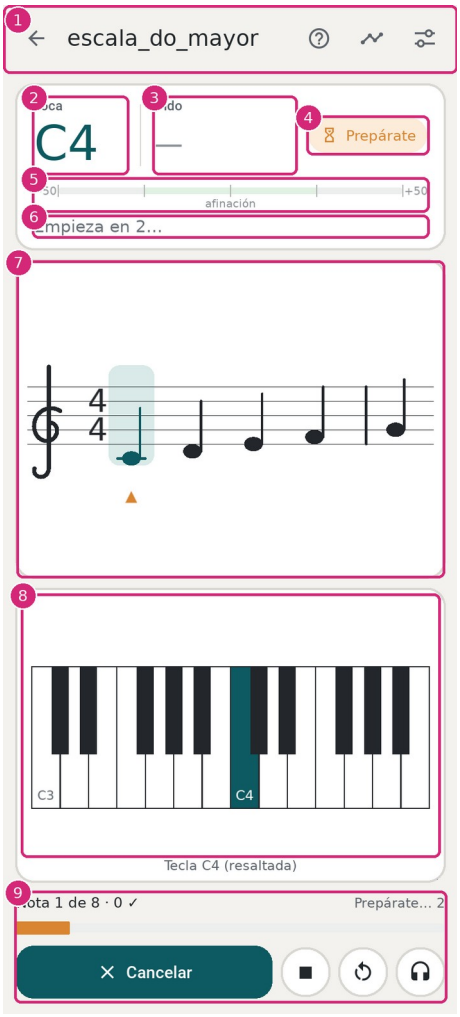


Figura 13**.** Cuenta previa («Prepárate… 2»)

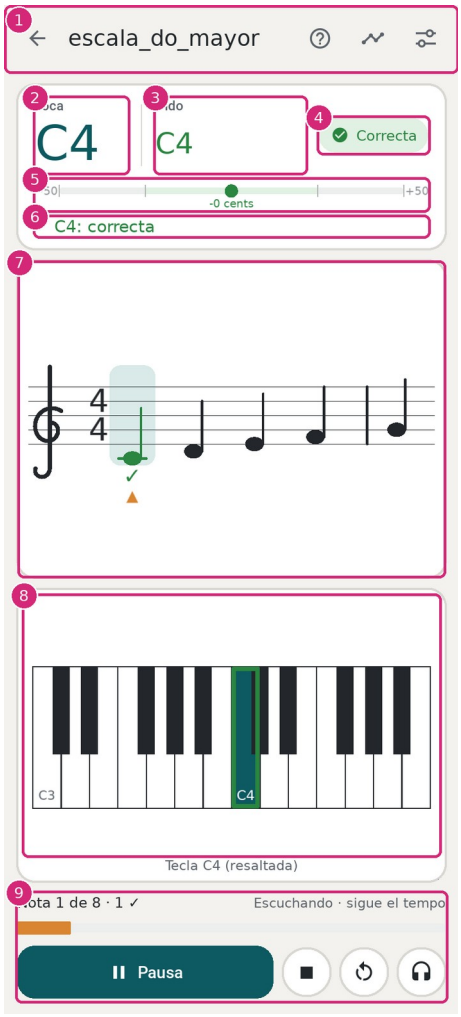


Figura 14**.** Nota correcta: C4 a +0 cents (✓ verde)

Captura de la aplicación: ventana de 412 × 915 dp (densidad 2), Linux x86_64, 04/11/2026; marcadores numerados añadidos con Pillow. Entrada simulada (ondas senoidales), sin micrófono.

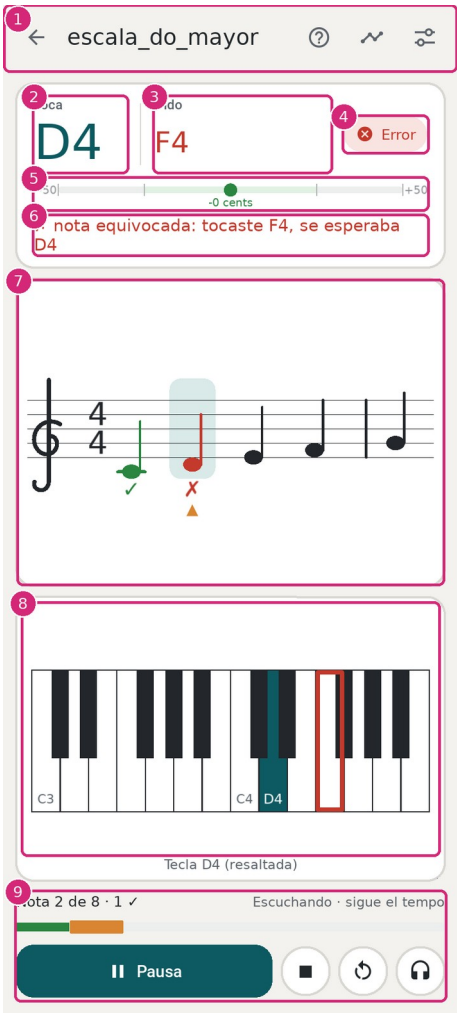


Figura 15. Nota incorrecta: F4 cuando se esperaba D4 (× rojo)

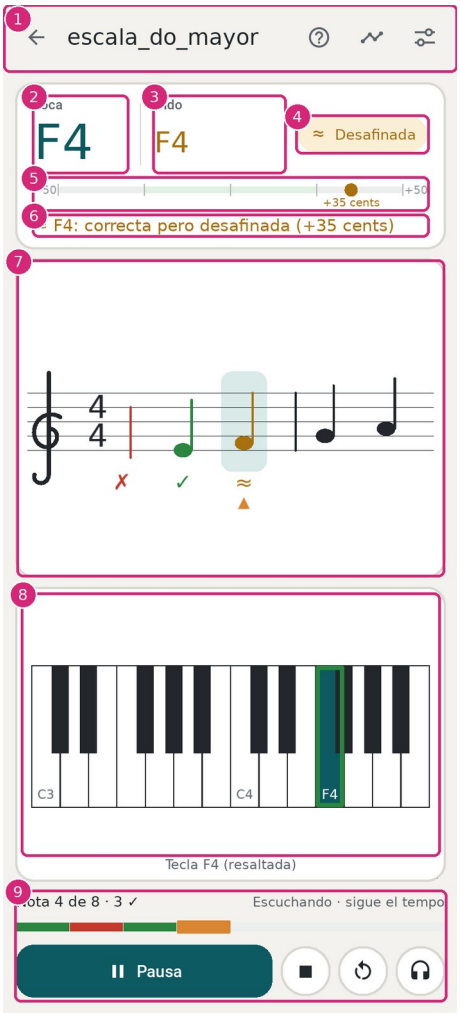


Figura 16. Nota desafinada: F4 a +35 cents (≈ ocre, cuenta como acierto)

Captura de la aplicación: ventana de 412 × 915 dp (densidad 2), Linux x86_64, 04/11/2026; marcadores numerados añadidos con Pillow. Entrada simulada (ondas senoidales), sin micrófono. Los números corresponden a la Tabla 25.

4.8 Resultados e historial

Al cerrarse la última ventana, la sesión termina, se registra en `progreso_sesiones.json` y aparece el resumen (Figura 17, Tabla 26). La pantalla `progreso` (Figura 18, Tabla 27) lee ese archivo; su gráfico «Últimas sesiones» lo dibuja `componentes.GraficoSesiones`.



Figura 17**.** Diálogo «Práctica terminada»: 6 de 8 notas correctas (75 %)

Captura de la aplicación: ventana de 412 × 915 dp (densidad 2), Linux x86_64, 04/11/2026; marcadores numerados añadidos con Pillow. Entrada simulada (ondas senoidales), sin micrófono.

Tabla 26**.** Elementos del resumen final

N°	Elemento	Qué hace	Archivo / clase
1	Resultado global	Porcentaje de aciertos sobre las notas evaluables.	`componentes.AnilloProgreso` · `WynikApp._mostrar_resumen_final`
2	Desglose	Correctas, errores de nota u octava, sin tocar o sin señal y acordes no evaluados.	`WynikApp._terminar` · `SesionDePractica.resumen`
3	Ver progreso	Abre la pantalla de progreso.	`change_screen("progreso")`
4	Repetir	Vuelve a empezar la misma práctica.	`WynikApp.boton_principal`

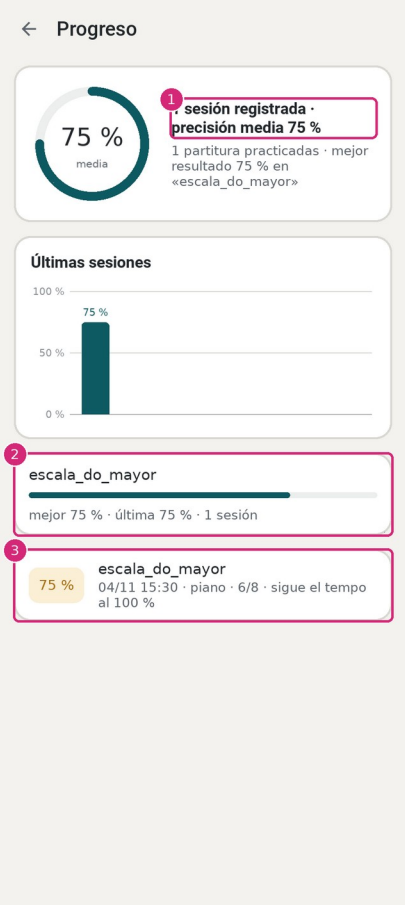


Figura 18**.** Pantalla «Progreso» con la sesión registrada

Captura de la aplicación: ventana de 412 × 915 dp (densidad 2), Linux x86_64, 04/11/2026; marcadores numerados añadidos con Pillow.

Tabla 27**.** Elementos de la pantalla de progreso

N°	Elemento	Qué hace	Archivo / clase
1	Resumen	Número de sesiones y precisión media, con el anillo de la media.	`WynikApp.refrescar_progreso` · `progreso_service.cargar_progreso`
2	Por partitura	Mejor y última precisión y número de sesiones de cada partitura.	`FilaPartituraProgreso` · `resumen_por_partitura`
3	Sesiones recientes	Fecha, instrumento, aciertos/total, modo y tempo de cada sesión.	`FilaSesion` · `progreso_sesiones.json`

4.9 Vista simplificada

Si en la preparación se elige la vista «Simplificada», la práctica usa la pantalla `partitura_simple` (Figura 19, Tabla 28), más ligera que el pentagrama completo.

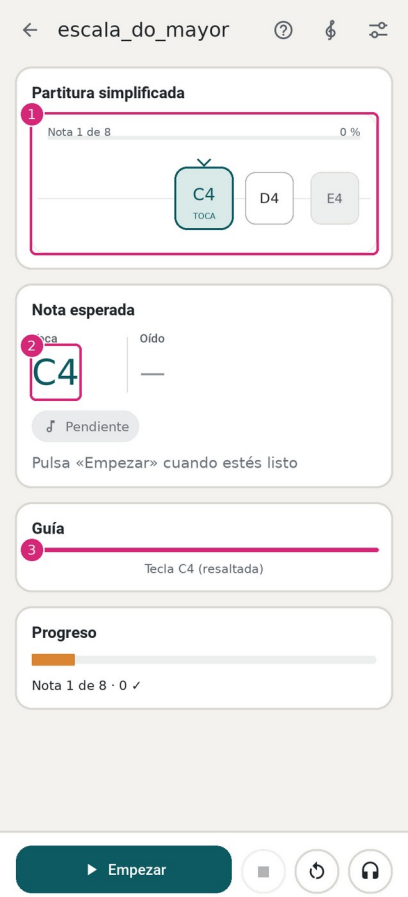


Figura 19**.** Práctica con la vista simplificada

Captura de la aplicación: ventana de 412 × 915 dp (densidad 2), Linux x86_64, 04/11/2026; marcadores numerados añadidos con Pillow.

Tabla 28**.** Elementos de la vista simplificada

N°	Elemento	Qué hace	Archivo / clase
1	Cinta de notas	Notas como fichas, con la actual resaltada y el porcentaje de avance.	`widgets.PartituraSimpleWidget`
2	Nota esperada	Nota que se debe tocar y estado.	`ids.nota_esperada` en <PartituraSimpleScreen>
3	Guía del instrumento	Indicación textual de la tecla o posición.	`GuiaInstrumentoWidget`

5. Manual del programador

5.1 Requisitos previos

La Tabla 29 reúne los requisitos de cada sistema. Los de macOS y Linux provienen de la cabecera de `requirements-escritorio.txt` y del README; el de PortAudio en Linux se comprobó durante la instalación en Ubuntu 24.04.

Tabla 29**.** Requisitos previos por sistema operativo

Sistema	Requisitos	Motivo
Todos	Python 3.13 o superior, Git y un micrófono para practicar	`requires-python`; captura de audio
macOS	`brew install cairo pkg-config`	pycairo (dependencia de KivyMD vía materialshapes) se compila
Linux (Debian/Ubuntu)	`libcairo2-dev`, `pkg-config`, compilador C y `libportaudio2`; en aarch64, además, compilador C++	pycairo y materialyoucolor se compilan; sin PortAudio, `import sounddevice` falla con «PortAudio library not found»
Windows 10/11 x64	Ninguno adicional	Todas las dependencias tienen rueda para Windows x64 (cabecera del `.txt`)

5.2 Instalación paso a paso

```
git clone https://github.com/MansillsRich/Wynik-app.git
cd Wynik-app
python3.13 -m venv .venv
source .venv/bin/activate            # Windows: .venv\Scripts\activate
python -m pip install --upgrade pip
python -m pip install -r requirements-escritorio.txt
```

En Ubuntu 24.04 la instalación se realizó con `uv pip install -r requirements-escritorio.txt` sobre Python 3.13.18 y resolvió los 37 paquetes no exclusivos de Windows, incluida la compilación de pycairo. En el computador de macOS, el entorno virtual se ubica fuera del repositorio (`../.venv`).

5.3 Ejecución

```
python main.py

# Datos en otra carpeta (pruebas, varias instalaciones):
WYNIK_DATOS=/ruta/a/datos python main.py
```

La ventana de escritorio abre en 1040 × 720 y se puede reducir hasta 360 × 340 (tamaño de teléfono apaisado). La aplicación no abre el micrófono al arrancar, sino al pulsar «Empezar»; en macOS, la primera vez el sistema pide permiso de micrófono para la aplicación desde la que se ejecuta Python (por ejemplo, Terminal).

5.4 Documentación del código

El código está documentado con docstrings en español. Según un recuento con el módulo `ast`, tienen docstring **50 de 54 módulos**, **73 de 110 clases** y **174 de 530 funciones y métodos**. Los docstrings de módulo explican el diseño (por ejemplo, las etapas de YIN o la política de estados de la sesión) y suelen citar los códigos de requisito C8–C11 y DET-01 a DET-05.

La documentación HTML se genera con **pydoc**, incluido en Python, sin dependencias nuevas. La carpeta `docs/` contiene 48 módulos y un índice generados con pydoc (commit 358dcaf). Para regenerarla se incluye `docs/anexo_iv/generar_pydoc.py`, que recorre los paquetes y escribe un HTML por módulo y un índice en `docs/anexo_iv/pydoc/` (Figura 20). En Linux sin pantalla se antepone `xvfb-run -a` al comando.

```
python docs/anexo_iv/generar_pydoc.py     # todos los módulos
python -m pydoc -w audio.estimador_yin    # un solo módulo
python -m pydoc -b                        # servidor local de documentación
```



Figura 20**.** Documentación generada con pydoc para el módulo `audio.estimador_yin`

Página `docs/anexo_iv/pydoc/audio.estimador_yin.html` abierta en Chromium.

5.5 Pruebas automatizadas

Las pruebas usan `unittest` (biblioteca estándar) y se ejecutan desde la raíz del repositorio. El 04/11/2026 las **13 pruebas** se ejecutaron con resultado correcto en Linux (Figura 21), macOS (Figura 23), Windows 11 (Figura 26) y Fedora Linux (Figura 27). La Tabla 30 indica qué verifica cada archivo.

```
python -m unittest discover -s tests -v
```

Tabla 30**.** Pruebas por archivo (resultado del 04/11/2026)

Archivo · clase	N°	Qué verifica	Resultado
`test_ajustes.py` · `TestAjustes`	2	Valores por defecto de `AjustesPractica` y corrección de valores fuera de rango (instrumento, La4, factor de tempo, tolerancia).	2/2 OK
`test_almacenamiento.py` · `TestAlmacenamiento`	4	Directorio de datos configurado, creación de `.cache`, escritura y lectura de JSON y lectura de un archivo inexistente.	4/4 OK
`test_musica.py` · `TestMusica`	4	`Altura.desde_nombre`, enarmónicos, `comparar_alturas` y nombre inválido.	4/4 OK
`test_progreso.py` · `TestProgreso`	3	`SesionProgreso.to_dict`/`from_dict`, persistencia con `registrar_sesion` y `registrar_sesion_practica`.	3/3 OK
Total	**13**	Tiempo de ejecución en Linux: 0,008 s	**13/13 OK**

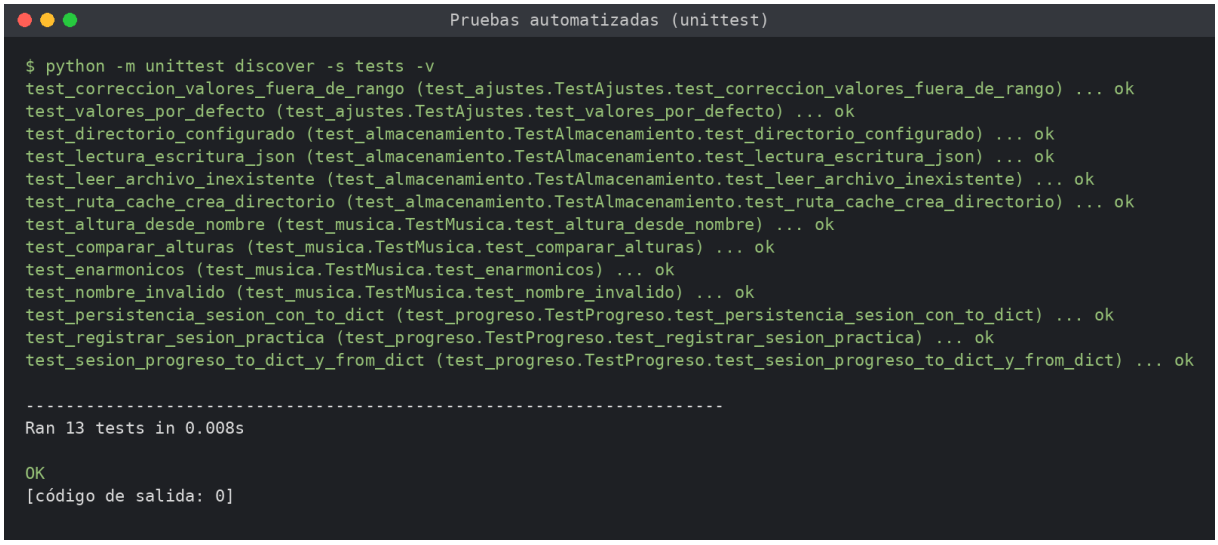


Figura 21**.** Salida de `python -m unittest discover -s tests -v` en Linux

Linux x86_64 (Ubuntu 24.04), Python 3.13.18, 04/11/2026.

La cobertura se midió con coverage.py 7.17.1 sobre los cuatro paquetes de la aplicación (Tabla 31). Las pruebas cubren el **10,1 %** de las sentencias y se concentran en `nucleo/` y `servicios/` (modelo musical, almacenamiento, ajustes y progreso). El detalle por archivo está en `evidencias/cobertura.txt`.

```
python -m pip install coverage
coverage run --source=audio,nucleo,servicios,vistas -m unittest discover -s tests
coverage report
```

Tabla 31**.** Cobertura de sentencias por paquete (coverage.py 7.17.1, 04/11/2026)

Paquete	Sentencias	Cubiertas	Cobertura	Módulos con cobertura > 0
audio/	982	0	0 %	Ninguno
nucleo/	1618	469	29,0 %	musica 63 %, instrumentos 69 %, eventos 66 %, sesion 39 %
servicios/	657	111	16,9 %	almacenamiento 85 %, ajustes_service 77 %, progreso_service 76 %
vistas/	2478	0	0 %	Ninguno
Total	**5735**	**580**	**10,1 %**	—

5.6 Calidad de código con Ruff

Ruff usa la configuración de `pyproject.toml` (reglas E y W de pycodestyle, F de Pyflakes e I de orden de importaciones; ignora E402 y E501). Con Ruff 0.17.3, `ruff check .` informa **37 avisos**, 35 de ellos con corrección automática (Tabla 32, Figura 22). El archivo `vistas/app.py` concentra 15.

```
python -m pip install ruff
ruff check .                # informe
ruff check . --statistics   # resumen por regla
ruff check . --fix          # aplica las correcciones automáticas
```

Tabla 32**.** Avisos de Ruff por regla (`ruff check . --statistics`)

Regla	Descripción	Avisos	Corrección automática
I001	Importaciones sin ordenar	18	Sí
F401	Importación sin usar	14	Sí
W292	Falta salto de línea al final del archivo	3	Sí
W291	Espacios al final de la línea	1	No (manual)
F841	Variable asignada y no usada	1	No (manual)
Total		**37**	35

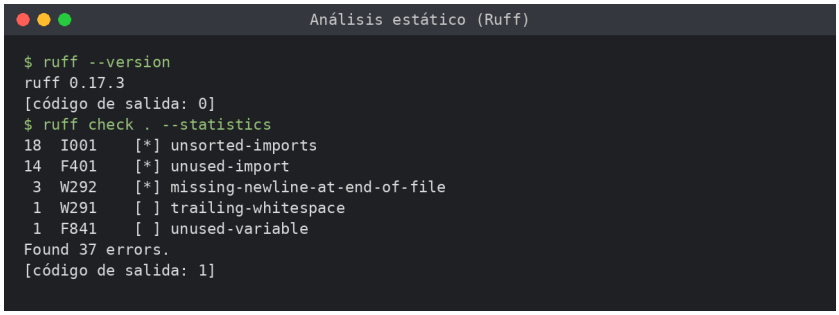


Figura 22**.** Salida de `ruff check . --statistics` (Ruff 0.17.3)

Linux x86_64 (Ubuntu 24.04), 04/11/2026.

5.7 Depuración

5.7.1 Registros

Kivy escribe su registro en la consola y en archivos dentro de `~/.kivy/logs/` (o de `KIVY_HOME/logs/` si se define esa variable). El nivel se cambia sin tocar el código con `KCFG_KIVY_LOG_LEVEL=debug python main.py`. La aplicación escribe sus propios mensajes en la consola con el prefijo `[Wynik]` (por ejemplo, «No se pudo abrir el micrófono», «No se pudo leer ajustes_wynik.json») y el historial usa `[HistorialService]`. `MotorDeEscucha.estadisticas()` y `ControladorPractica.resumen()` devuelven bloques descartados, desbordes, discontinuidades, carga del análisis y el último error del hilo.

5.7.2 Probar sin micrófono

La arquitectura permite depurar el análisis sin micrófono:

- `Analizador(fs, perfil).analizar_senal(señal)` analiza una señal completa y devuelve los eventos (así se obtuvo la Tabla 15).
- `MotorDeEscucha(..., captura=CapturaManual(fs), usar_hilo=False)` y `procesar_pendientes()` hacen el análisis determinista, sin hilos.
- `WynikApp.fabrica_captura` sustituye el micrófono por cualquier captura con la misma interfaz; `capturar_pantallas.py` lo usa para la entrada simulada.

5.7.3 Problemas típicos

La Tabla 33 resume los problemas conocidos, su causa y su solución.

Tabla 33**.** Problemas típicos, causa y solución

Síntoma	Causa	Solución
Diálogo «No se pudo abrir el micrófono» al pulsar Empezar	`MicrofonoNoDisponibleError`: sin permiso, sin dispositivo o dispositivo ocupado	macOS: Ajustes del Sistema → Privacidad y seguridad → Micrófono → activar la aplicación desde la que se ejecuta Python; revisar el dispositivo en Ajustes («Cambiar micrófono»). Sin permiso, macOS también puede entregar silencio sin error.
«No se encontró ningún micrófono» en Ajustes (Linux)	sounddevice no encuentra PortAudio («PortAudio library not found»)	Instalar `libportaudio2`.
Falla la instalación de pycairo	pycairo se compila y necesita cairo	macOS: `brew install cairo pkg-config`; Linux: `libcairo2-dev pkg-config`.
Interbloqueo al cargar la guía en Linux	Filtro de eventos de SDL2 frente a PulseAudio/PipeWire	Resuelto en `main.py`: inicializa los proveedores de `SoundLoader` antes de crear la ventana.
Aviso «too much iteration» al redimensionar	Las tarjetas se reajustan en más de 20 pasadas del reloj	Resuelto: `Clock.max_iteration = 40` en `WynikApp.build`.
Textos y botones duplicados; «Ignored class … re-declaration»	`wynik.kv` se cargaba dos veces	Resuelto: `WynikApp.load_kv` devuelve False y `build()` carga los KV una vez.
Avisos emergentes que no aparecen en macOS	Fbo de alto 0 incompleto en OpenGL sobre Metal	Resuelto: `vistas/avisos.py` (`SnackbarWynik`).

5.8 Plataformas

El desarrollo se realizó en macOS, Windows 11 y Fedora Linux, y la aplicación se ejecutó en los tres sistemas. La Tabla 34 resume el estado de cada plataforma y su evidencia.

Tabla 34**.** Estado de cada plataforma con su evidencia

Plataforma	Estado	Evidencia o motivo
Linux x86_64 (Ubuntu 24.04)	**Validado** el 04/11/2026: instalación, 13/13 pruebas, Ruff, pydoc, interfaz completa y práctica con entrada simulada	`evidencias/`; capítulo 4 (sin micrófono físico)
macOS 27.2 «Golden Gate» (MacBook Air 15″, Apple M5, 16 GB)	**Validado** el 04/11/2026: 13/13 pruebas, ejecución de la aplicación y práctica con piano real y micrófono	Figura 23 a Figura 25; `.venv` con Python 3.13.16 y los 37 paquetes fijados
Windows 11 x64	**Validado**: 13/13 pruebas el 04/11/2026; aplicación ejecutada durante el desarrollo	Figura 26; Python 3.12.3 de 64 bits (AMD64)
Windows 11 ARM (VMware Fusion en el MacBook Air M5)	Probado por el equipo	Las ruedas declaradas en el `.txt` son para Windows x64
Fedora Linux x86_64	**Validado**: 13/13 pruebas el 04/11/2026; aplicación ejecutada durante el desarrollo	Figura 27; Python 3.14.10; requiere cairo, pkg-config y PortAudio (apartado 5.1)
Linux aarch64 (Kali en UTM)	Soportado por dependencias, no validado	Cabecera del `.txt`: además de pycairo, se compila materialyoucolor (necesita C++)
Android	En desarrollo: APK de prueba	APK de prueba generado con Buildozer, con errores de compatibilidad entre versiones en resolución (Tabla 35)
iOS	No soportado	Sin código ni empaquetado específicos

5.8.1 Prueba en macOS con piano real

El 04/11/2026 se ejecutaron las pruebas automatizadas en el entorno virtual del MacBook Air (Figura 23: 13 pruebas, OK) y se practicó la partitura «Hjärtats_saga» (121 notas) en el modo «Esperar cada nota», con un piano real y el micrófono del computador. En una nota que pedía La4, primero se tocó una nota equivocada y la aplicación la marcó como error: la Figura 24 muestra Do♯1 en «Oído» y el estado «Error». El mensaje «octava incorrecta: tocaste La2» corresponde a un intento anterior de esa misma nota, porque el mensaje solo cambia cuando un intento modifica el resultado. Luego se volvió a intentar con la nota correcta y la aplicación la aceptó: la Figura 25 muestra el La4 reconocido como correcto a −18 cents, dentro de la tolerancia de ±25 cents. La detección distinguió así la nota pedida de una distinta.

```
Last login: Tue Nov  3 21:03:22 on console
richardmansilla@MacBook-Air-de-Richard ~ %    cd ~/Documents/Projects/"Wynik test_01"/Wynik-app
[   source ../.venv/bin/activate                                                                  ]
[(.venv) richardmansilla@MacBook-Air-de-Richard Wynik-app %    python -m unittest discover -s tests -v ]
test_correccion_valores_fuera_de_rango (test_ajustes.TestAjustes.test_correccion_valores_fuera_de_rango) ... ok
test_valores_por_defecto (test_ajustes.TestAjustes.test_valores_por_defecto) ... ok
test_directorio_configurado (test_almacenamiento.TestAlmacenamiento.test_directorio_configurado) ... ok
test_lectura_escritura_json (test_almacenamiento.TestAlmacenamiento.test_lectura_escritura_json) ... ok
test_leer_archivo_inexistente (test_almacenamiento.TestAlmacenamiento.test_leer_archivo_inexistente) ... ok
test_ruta_cache_crea_directorio (test_almacenamiento.TestAlmacenamiento.test_ruta_cache_crea_directorio) ... ok
test_altura_desde_nombre (test_musica.TestMusica.test_altura_desde_nombre) ... ok
test_comparar_alturas (test_musica.TestMusica.test_comparar_alturas) ... ok
test_enarmonicos (test_musica.TestMusica.test_enarmonicos) ... ok
test_nombre_invalido (test_musica.TestMusica.test_nombre_invalido) ... ok
test_persistencia_sesion_con_to_dict (test_progreso.TestProgreso.test_persistencia_sesion_con_to_dict) ... ok
test_registrar_sesion_practica (test_progreso.TestProgreso.test_registrar_sesion_practica) ... ok
test_sesion_progreso_to_dict_y_from_dict (test_progreso.TestProgreso.test_sesion_progreso_to_dict_y_from_dict) ... ok

----------------------------------------------------------------------
Ran 13 tests in 0.002s

OK
(.venv) richardmansilla@MacBook-Air-de-Richard Wynik-app %
```

Figura 23**.** Pruebas automatizadas en macOS: 13 pruebas, OK

MacBook Air 15" (Apple M5, macOS 27.2), entorno `../.venv` con Python 3.13.16, 04/11/2026.
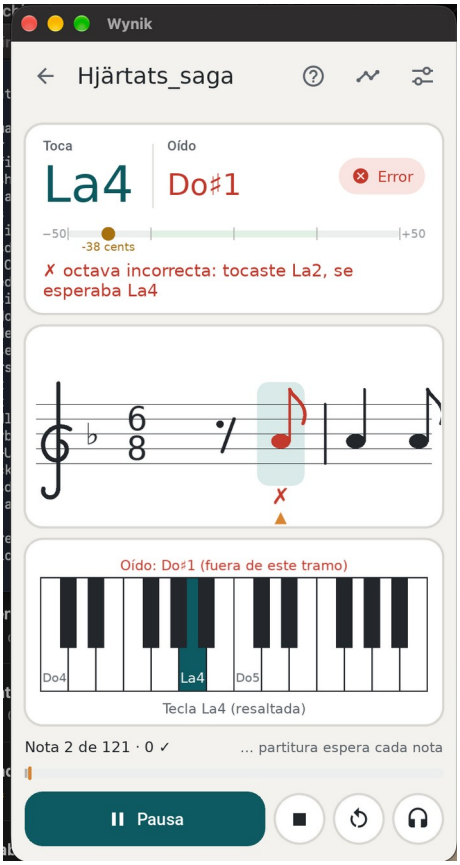


Figura 24**.** macOS con piano real, primer intento: Do♯1 cuando se esperaba La4, marcado como error

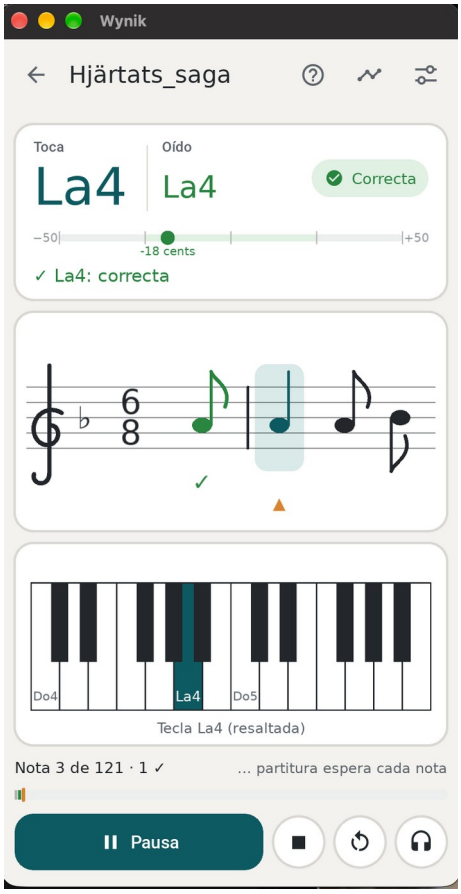


Figura 25**.** macOS con piano real, segundo intento: La4 correcto a −18 cents

macOS, 04/11/2026: partitura «Hjärtats_saga», modo «Esperar cada nota», micrófono del MacBook Air y piano real.

5.8.2 Pruebas en Windows 11 y Fedora Linux

En Windows 11 y en Fedora Linux se ejecutó `python -m unittest discover -s tests -v` desde la raíz del repositorio, con 13 pruebas correctas en ambos sistemas. La versión y la arquitectura de Python se obtuvieron con `python -c "import platform, sys; print(sys.version, platform.machine())"`: Python 3.12.3 de 64 bits (AMD64) en Windows 11 (Figura 26) y Python 3.14.10 sobre x86_64 en Fedora Linux (Figura 27). Las pruebas también se ejecutan correctamente con Python 3.12.3, aunque `pyproject.toml` declara como versión mínima la 3.13.

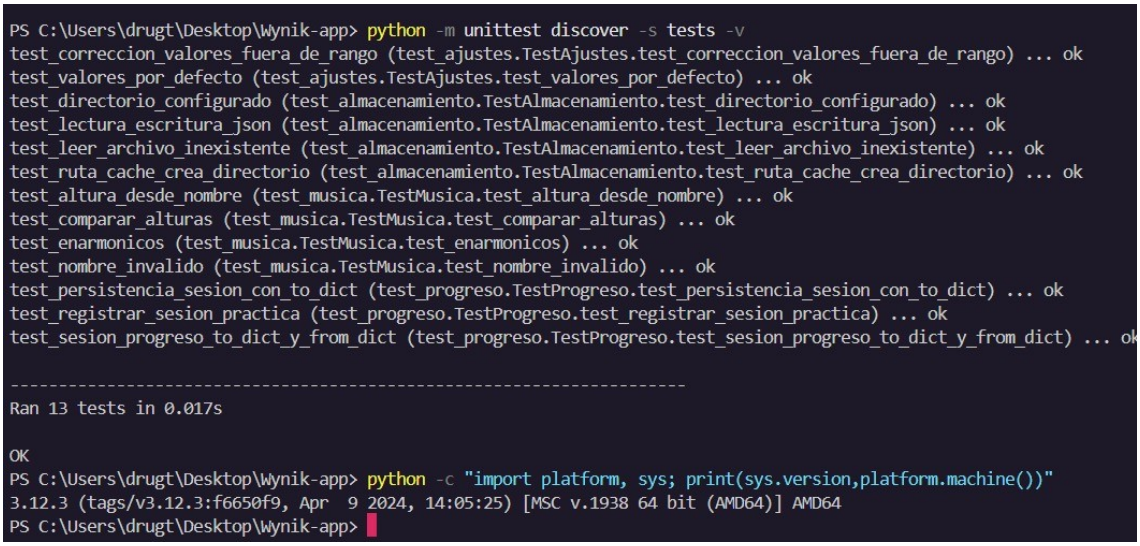


Figura 26**.** Pruebas automatizadas en Windows 11: 13 pruebas, OK, con Python 3.12.3 (AMD64)

PowerShell en Windows 11, 04/11/2026.

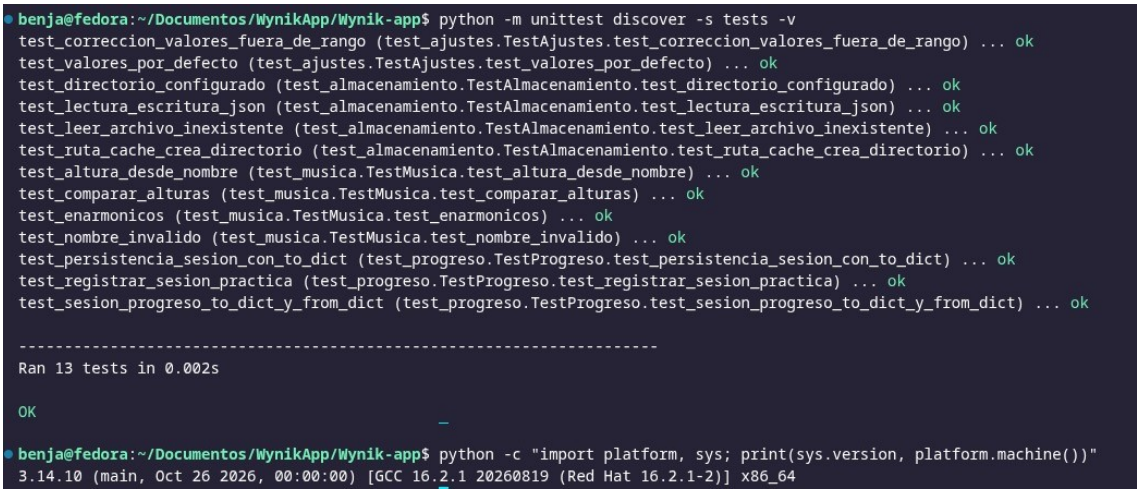


Figura 27**.** Pruebas automatizadas en Fedora Linux: 13 pruebas, OK, con Python 3.14.10 (x86_64)

Terminal de Fedora Linux, 04/11/2026 (dos capturas consecutivas unidas).

5.8.3 Android

La versión para Android se empaqueta con Buildozer como un APK de prueba. En la etapa actual presenta errores de compatibilidad entre versiones de dependencias, en cuya resolución trabaja el equipo, y aún no reconoce correctamente el micrófono ni las carpetas para seleccionar partituras. La Tabla 35 resume el estado de cada aspecto.

Tabla 35**.** Estado de Wynik en Android

Aspecto	Estado actual
Empaquetado	APK de prueba generado con Buildozer; presenta errores de compatibilidad entre versiones de dependencias, en los que el equipo está trabajando
Captura de audio	`audio/captura.py` prevé un adaptador `AudioRecord` con pyjnius y la misma interfaz que `CapturaBase`; en el APK de prueba el micrófono aún no se reconoce correctamente
Permiso de micrófono	El código de la aplicación no solicita `RECORD_AUDIO` en tiempo de ejecución
Análisis	`audio/` solo usa NumPy; `nucleo/`, la biblioteca estándar y music21 (al leer MusicXML)
Datos locales	`configurar_directorio(app.user_data_dir)` en Android
Selector de archivos	`FileChooserListView` sobre la carpeta personal; en el APK de prueba aún no accede correctamente a las carpetas de partituras
Guía de audio	`SoundLoader` de Kivy, el mismo código que en escritorio
Prueba	APK de prueba ejecutado por el equipo

6. Estado de funcionalidades y trazabilidad

6.1 Mecanismos de control de calidad aplicados

Siguiendo el apartado 5.4 de la plantilla CCII, la Tabla 36 enumera los mecanismos usados para asegurar la calidad del código y de este documento.

Tabla 36**.** Mecanismos de control de calidad

Mecanismo	Aplicación	Resultado
Pruebas automatizadas (unittest)	`python -m unittest discover -s tests -v`	13/13 correctas en Linux, macOS, Windows 11 y Fedora Linux
Medición de cobertura (coverage.py)	Sobre audio, nucleo, servicios y vistas	10,1 %
Análisis estático (Ruff)	`ruff check .` con la configuración del proyecto	37 avisos, 35 automáticos
Verificación de versiones	`pyproject.toml`, `.in`, `.txt` y entornos instalados	Coinciden las cinco dependencias directas
Prueba manual con instrumento real	macOS, piano real y micrófono, 04/11/2026	Nota correcta e incorrecta detectadas
Ejecución de la interfaz y capturas reproducibles	`capturar_pantallas.py` y `anotar_capturas.py`; revisión de cada imagen	16 capturas
Revisión del documento	Contraste de la versión 2.0 con el código (`docs/anexo_iv/diagnostico.md`)	Correcciones incorporadas en la versión 3.0
Integridad documental	Índices automáticos, figuras y tablas numeradas y citadas, fecha y versión iguales en portada, ficha y pie	Verificado en el PDF final

6.2 Matriz de trazabilidad

La Tabla 37 enlaza cada funcionalidad con el código que la implementa, la prueba automatizada que la cubre y la evidencia. «Sin prueba automatizada» indica que la funcionalidad se verificó mediante ejecución. Los códigos C8–C11 y DET-xx corresponden a los requisitos citados en los docstrings.

Tabla 37. Trazabilidad funcionalidad → implementación → prueba → evidencia

Funcionalidad	Módulo / clase	Prueba que la cubre	Evidencia	Estado
Importar partituras MusicXML	`nucleo.importador_musicxml.importar_musicxml`	Sin prueba automatizada	Figura 6, Figura 8	Implementado
Biblioteca e historial de partituras	`servicios.historial_service`, `ElegirPartituraScreen`	Sin prueba automatizada	Figura 7	Implementado
Captura de audio de escritorio	`audio.captura.CapturaSounddevice`	Sin prueba automatizada	Figura 24, Figura 25	Validado en macOS (piano real)
Detección de notas (YIN, segmentación)	`audio.analizador`, `estimador_yin`, `segmentador`	Sin prueba automatizada	Tabla 15; Figura 14–Figura 16; Figura 25	Simulada y en macOS con piano
Comparación con la nota esperada (cents)	`nucleo.musica.comparar_frecuencia`	`test_musica` (solo `comparar_alturas`)	Tabla 13, Tabla 15	Implementado
Sincronización con el tempo (C8, DET-01, DET-02)	`nucleo.sesion.SesionDePractica`	Sin prueba automatizada	Tabla 15	Implementado
Política de estados y acordes (C9, DET-03, DET-05)	`SesionDePractica`, `EstadoNota`	Sin prueba automatizada	Figura 15	Implementado
Retroalimentación visual en tiempo real	`vistas.app.WynikApp`, `PartituraWidget`, `GuiaInstrumentoWidget`	Sin prueba automatizada	Figura 14–Figura 16	Implementado
Guía de audio, nunca a la vez que la escucha (C10, C11)	`audio.guia`, `servicios.reproductor_guia`	Sin prueba automatizada	Figura 8	Implementado
Ajustes de tempo, modo y evaluación	`servicios.ajustes_service`, `AjustesInicioScreen`, `AjustesScreen`	`test_ajustes` (2)	Figura 9, Figura 11	Implementado
Persistencia local JSON	`servicios.almacenamiento`	`test_almacenamiento` (4)	Tabla 16	Implementado
Registro de sesiones y progreso	`servicios.progreso_service`, `SesionProgreso`	`test_progreso` (3)	Figura 17, Figura 18	Implementado
Vista simplificada	`PartituraSimpleScreen`, `PartituraSimpleWidget`	Sin prueba automatizada	Figura 19	Implementado
Ejecución en Android	Ganchos en `main.py` y `vistas/app.py`; APK de prueba con Buildozer	—	Tabla 35	En desarrollo
Importar PDF mediante OMR	—	—	—	Excluido del MVP

7. Criterios de aceptación y exclusiones

7.1 Criterios de aceptación del MVP

La Tabla 38 presenta los criterios de aceptación del MVP y la evidencia de su cumplimiento.

Tabla 38**.** Criterios de aceptación y su cumplimiento

Criterio	Estado	Evidencia
Ejecutar la aplicación sin registro ni inicio de sesión	Cumplido	Capítulo 4; no hay autenticación en el código
Importar partituras MusicXML	Cumplido	Figura 6, Figura 8
Capturar audio con el micrófono y analizar la altura en escritorio	Cumplido en macOS con piano real (04/11/2026)	Tabla 15; Figura 24, Figura 25
Mostrar la retroalimentación durante la práctica	Cumplido	Figura 14–Figura 16
Guardar el progreso en JSON local	Cumplido	Tabla 16; Figura 18
Pasar la suite de pruebas automatizadas	Cumplido (13/13 en Linux, macOS, Windows 11 y Fedora Linux), con cobertura del 10 %	Tabla 30, Tabla 31; Figura 26, Figura 27

7.2 Exclusiones del MVP

- Procesamiento de partituras PDF mediante reconocimiento óptico de música (OMR).
- Servicios en la nube, autenticación remota o bases de datos externas.
- Funciones de red para la interacción entre profesores y alumnos.
- Análisis polifónico: los acordes se muestran como referencia y no se evalúan (DET-03).
- Empaquetado móvil definitivo (APK para Android o paquete para iOS).
- Medición sistemática de la tasa de aciertos del detector con instrumentos reales.

8. Definiciones y abreviaturas

La Tabla 39 define los términos y abreviaturas usados en este anexo.

Tabla 39**.** Definiciones y abreviaturas

Término	Definición
APK	Paquete de instalación de una aplicación Android.
Ataque (onset)	Inicio de una nota: subida brusca de energía o cambio del espectro.
AVD	Android Virtual Device: emulador de Android.
BPM	Pulsos por minuto; en Wynik, negras por minuto (♩ = 80).
Buildozer / python-for-android	Herramientas para empaquetar aplicaciones Kivy como APK.
CCII	Consejo General de Colegios Profesionales de Ingeniería en Informática (España), autor de la plantilla usada.
Cent	Centésima de semitono; una octava tiene 1200 cents.
CMND	Diferencia media acumulada normalizada d′(τ), paso 3 del algoritmo YIN.
Cobertura	Porcentaje de sentencias del código que ejecutan las pruebas.
dBFS	Decibelios respecto de la escala completa digital (0 dBFS = valor máximo).
Docstring	Cadena de documentación de un módulo, clase o función de Python.
dp	Píxel independiente de la densidad, unidad de medida de Kivy.
FFT	Transformada rápida de Fourier.
Flujo espectral	Aumento de energía del espectro entre dos tramas; sirve para detectar ataques.
Hilo (thread)	Flujo de ejecución concurrente dentro del proceso.
JSON	JavaScript Object Notation: formato de texto para datos estructurados.
KV	Lenguaje declarativo de Kivy para describir interfaces.
La4	Nota La de la cuarta octava; referencia de afinación (440 Hz por defecto).
MIDI (número)	Número entero de una altura temperada: C4 = 60, A4 = 69.
MusicXML / MXL	Formato XML estándar para partituras; MXL es su versión comprimida.
MVP	Producto mínimo viable.
OMR	Reconocimiento óptico de música (partituras en imagen o PDF).
PortAudio	Biblioteca de entrada y salida de audio multiplataforma que usa sounddevice.
pydoc	Generador de documentación de la biblioteca estándar de Python.
PyPI	Python Package Index, repositorio público de paquetes de Python.
RMS	Valor cuadrático medio de una señal (nivel).
Rueda (wheel)	Paquete de Python ya compilado para una plataforma.
Ruff	Analizador estático y formateador de Python.
SDL2	Biblioteca multimedia que Kivy usa para la ventana, la entrada y el audio.
SuperFlux	Variante del flujo espectral con filtro de máximos que reduce los falsos ataques por vibrato.
UCT	Universidad Católica de Temuco.
unittest	Marco de pruebas unitarias de la biblioteca estándar de Python.
venv	Entorno virtual de Python con sus propias dependencias.
Xvfb	Servidor X virtual que permite ejecutar aplicaciones gráficas sin pantalla.
YIN	Algoritmo de estimación de la frecuencia fundamental de de Cheveigné y Kawahara (2002).

9. Bibliografía

Astral Software. (s. f.). *Ruff documentation*. Recuperado el 4 de noviembre de 2026, de https://docs.astral.sh/ruff/

Astral Software. (s. f.). *uv documentation*. Recuperado el 4 de noviembre de 2026, de https://docs.astral.sh/uv/

Batchelder, N. (s. f.). *Coverage.py* (versión 7.17.1) [Software]. https://coverage.readthedocs.io/

Böck, S., & Widmer, G. (2013). Maximum filter vibrato suppression for onset detection. En *Proceedings of the 16th International Conference on Digital Audio Effects (DAFx-13)*. Maynooth, Irlanda.

Consejo General de Colegios Profesionales de Ingeniería en Informática. (2016). *Plantilla básica para la elaboración de la documentación de proyectos de ingeniería informática* (Norma CCII-N2016-02, versión 1.0).

Cuthbert, M. S., & Ariza, C. (2010). music21: A toolkit for computer-aided musicology and symbolic music data. En *Proceedings of the 11th International Society for Music Information Retrieval Conference (ISMIR 2010)* (pp. 637–642).

de Cheveigné, A., & Kawahara, H. (2002). YIN, a fundamental frequency estimator for speech and music. *The Journal of the Acoustical Society of America, 111*(4), 1917–1930. https://doi.org/10.1121/1.1458024

Geier, M. (s. f.). *python-sounddevice* (versión 0.5.6) [Software]. https://python-sounddevice.readthedocs.io/

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., Fernández del Río, J., Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature, 585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Kivy Organization. (s. f.). *Kivy documentation* (versión 2.3.1). Recuperado el 4 de noviembre de 2026, de https://kivy.org/doc/stable/

KivyMD Team. (s. f.). *KivyMD documentation* (versión 2.0.0). Recuperado el 4 de noviembre de 2026, de https://kivymd.readthedocs.io/

PortAudio. (s. f.). *PortAudio: Portable cross-platform audio I/O*. Recuperado el 4 de noviembre de 2026, de https://www.portaudio.com/

Python Software Foundation. (s. f.). *pydoc — Documentation generator and online help system*. Python 3.13 documentation. https://docs.python.org/3.13/library/pydoc.html

Python Software Foundation. (s. f.). *unittest — Unit testing framework*. Python 3.13 documentation. https://docs.python.org/3.13/library/unittest.html

Rodríguez Ramírez, B. M. (2019). *Aikén: Aplicación móvil para turismo social. Anexo IV – Documentación técnica de programación* [Trabajo de fin de grado, Universidad de Salamanca].

W3C Music Notation Community Group. (2021). *MusicXML 4.0* [Final community group report]. https://www.w3.org/2021/06/musicxml40/

Firmas

Los integrantes del equipo de desarrollo suscriben el contenido de este anexo, que describe el estado del repositorio en el commit 358dcaf.

Segundo Richard Nicolás Mansilla Lleucún
Scrum Master y desarrollador

Sebastián Eduardo Contreras Vergara
Desarrollador y tester

Benjamín Ignacio Saavedra Mercado
Desarrollador y tester

Javiera Alejandra Celia Muñoz Toledo
Desarrolladora y tester

Temuco, 04/11/2026.

Declaración de uso de inteligencia artificial

El equipo declara el uso de herramientas de inteligencia artificial en el proyecto, siempre bajo supervisión humana. En el desarrollo de la aplicación se utilizaron como apoyo para escribir el código. En la elaboración de esta versión del anexo se utilizó un asistente de inteligencia artificial (Claude, de Anthropic) como apoyo para contrastar la documentación con el código, ejecutar los comandos de verificación (pruebas, cobertura, Ruff y pydoc), generar las capturas mediante scripts auxiliares que no modifican la aplicación y redactar borradores de las secciones.

Los datos del documento provienen del repositorio y de las salidas guardadas en `docs/anexo_iv/evidencias/`. El equipo revisó la totalidad del contenido y asume la responsabilidad sobre él.